

DEDAN KIMATHI UNIVERSITY OF TECHNOLOGY
CENTRE FOR DATA SCIENCE AND ARTIFICIAL INTELLIGENCE (DSAIL)

JasinaHub

Architecture and Developer Guide for Scalable Speech and Text Data Collection in Low-Resource African Languages

Braxton Mandara¹ Mary Kariuki² Ciira wa Maina³

^{1,2,3} Dedan Kimathi University of Technology,
Centre for Data Science and Artificial Intelligence (DSAIL),
P.O Box 657 – 10100, Nyeri, Kenya

¹braxton.mandara@dkut.ac.ke

²mary.kariuki@dkut.ac.ke

³ciira.maina@dkut.ac.ke

ABSTRACT

Many African languages lack the reviewed speech and transcription datasets needed to develop automatic speech recognition (ASR). JasinaHub helps communities create these datasets through a mobile-responsive web platform. Volunteers record natural responses to structured questions or read supplied sentences. Approved transcribers write what they hear, and administrators review the audio and text before exporting accepted pairs. The open-source, serverless platform controls access by role, reserves recordings for one transcriber at a time, and supports recovery of interrupted uploads and transcription drafts. Exported records use pseudonymous contributor identifiers instead of names.

Development used iterative AI-assisted prototyping with developer oversight. An operational pilot involved 25 consenting adults representing four regional Kikuyu varieties and used 369 health-related prompts. Participants submitted 2,275 recordings totalling 16 h 18 min. Following recording and transcript review, the dataset contained 1,789 completed speech–transcription pairs totalling 13 h 16 min: 78.6% of submitted recordings and 81.4% of recorded duration.

The pilot demonstrated the collection-to-export workflow under practical deployment conditions; it was not a formal usability study or an ASR evaluation. Its small sample, single language, and health-domain focus limit generalisation. The corpus is being used to develop *JasinaH*, a separate Kikuyu ASR model. JasinaHub is released under the MIT licence and supports configuration for additional languages and domains. Model development remains outside the platform itself.

Keywords: Speech corpus, Low-resource languages, Automatic Speech Recognition, Voice data collection, Transcription, Kikuyu, Progressive Web Application, PostgreSQL Row-Level Security, Crowdsourcing.

List of Figures

1	System architecture of JasinaHub illustrating how different elements interact with each other	9
2	End-to-end system flowchart showing speech data collection, moderation, transcription, verification, and dataset export workflows.	11
3	Use case diagram showing the three actors (Volunteer, Transcriber, Administrator) and the fourteen primary use cases of the JasinaHub system.	13
4	Entity-relationship diagram of the JasinaHub schema showing core entities, their attributes, and cardinality relationships. The <code>voice_responses</code> entity is the central hub linking volunteers, prompts, and transcriptions.	18
5	Sequence diagram for the atomic transcription claiming and submission process with pessimistic locking.	21
6	Sequence diagram of the voice recording pipeline from category selection through audio capture, local recovery backup, cloud upload, and database persistence.	22
7	Mobile volunteer recording interface in release v1.0, showing a Kikuyu prompt, active recording timer, stop control, prompt progress, and skip action.	23
8	Administrator command-centre overview showing collection statistics, action queues, navigation to management modules, and a response trend chart. Snapshot counts reflect the state of the running platform when captured.	29
9	Transcription workspace showing inline instructions, guideline access, the original Kikuyu prompt, audio playback controls, the verbatim text field, and submission action.	30
10	Administrator feedback-question management interface showing ordered questions, response types, required flags, activation controls, and editing actions.	32
11	Administrator test-runner interface showing point-in-time pass rate, executed-test coverage, destructive-test control, and individual Smoke and Authentication/Authorization checks.	35

List of Tables

1	Recommended reading paths by audience.	1
2	Document control information.	2
3	JasinaHub code and support metadata.	3
4	Required application-repository artifacts.	5
5	Environment-variable reference contract.	5
6	Comparison with related speech-collection platforms and workflows. A dash denotes a capability not reported in the cited publication, not evidence that it is absent from later software.	8
7	Minimum authorization matrix.	12
8	Feature-to-implementation traceability template.	15
9	Tables comprising the JasinaHub schema.	16
10	Transcription lock lifecycle.	21
11	Required recording-pipeline recovery behavior.	23
12	Documented RPC behaviour and required failure handling; exact signatures require release verification.	24
13	Recording rejection criteria used during administrative quality review.	29
14	JasinaHub pilot deployment and quality-review outcomes.	33
15	Required automated test layers.	35
16	Workflow summary and authoritative documentation.	37

Listings

1	Reference clean-checkout installation workflow.	4
2	Creating local configuration.	5
3	Reference local Supabase initialization.	6
4	Reference frontend verification workflow.	6
5	Recommended application repository layout.	14
6	Illustrative constraint pattern; adapt names to the canonical migrations.	17
7	Illustrative export manifest record.	31

Contents

Start Here: Reading Paths	1
1 Document Control and How to Use This Guide	2
1.1 Source-of-Truth Policy	2
1.2 Publication Alignment Policy	2
1.3 Privacy-Policy Alignment	2
1.4 Code Metadata	3
1.5 Terminology and Requirement Language	3
2 Introduction	3
3 System Scope and Assumptions	3
4 Developer Quick Start and Reproducibility	4
4.1 Prerequisites	4
4.2 Obtain and Install the Application	4
4.3 Required Repository Contract	5
4.4 Environment Configuration	5
4.5 Start the Local Backend	6
4.6 Run and Verify the Frontend	6
4.7 Bootstrap Test Accounts and the First Administrator	6
4.8 Clean-Checkout Acceptance Criteria	6
5 System Objectives and Design Principles	6
5.1 Primary Objectives	7
5.2 Architectural Principles	7
6 Related Work and Literature Review	7
7 System Architecture and Design	8
7.1 Technology Stack	9
7.2 High-Level Architecture	9
7.3 Behavioural Model	10
8 System Roles and Authorisation Model	12
8.1 Volunteer (Recording Contributor)	12
8.2 Transcriber	12
8.3 Administrator	12
8.4 Authorization Matrix	12
9 Development Process and Implementation Approach	13
9.1 Scalability Precautions	14
9.2 Expected Source-Code Organisation	14
9.3 Feature-to-Implementation Traceability	14
9.4 Contribution Workflow	15
10 Database Design	15
10.1 Schema Overview	16
10.2 Schema Reference Requirements	16
10.3 Migration and Seed Policy	17
10.4 Entity-Relationship Diagram	18
10.5 Row-Level Security Highlights	18
10.6 RLS and Privileged-Function Review Checklist	19

11 Domain Rules and Invariants	19
11.1 Configurable Recording-Duration Limit	19
11.2 One Recording per Contributor per Question	19
11.3 Question Retirement Threshold	19
11.4 Configurable Contributor Progress	20
11.5 Verbatim Transcription Policy	20
11.6 Configurable Correction Allowance	20
11.7 Single-Transcriber Editing Rights	20
12 Concurrency and Consistency Management	20
13 Voice Recording Pipeline	22
13.1 Audio Contract	22
13.2 Recording Failure Matrix	23
14 Backend Interface Reference	24
14.1 Core RPC Contracts	24
14.2 Edge Function Contracts	24
15 Storage Strategy and Migration	24
15.1 Storage Authorization and Object Lifecycle	25
15.2 Cloud Providers and Transfer Review	25
15.3 Retention and Deletion	25
15.4 Migration Procedure	25
16 Offline Resilience and Recovery	25
16.1 Browser Storage and Privacy	26
16.2 Recovery Record Contract	26
17 Security and Access Control	26
17.1 Threat Model and Security Boundaries	27
17.2 Privacy Policy, Controller, and Consent	27
17.3 Information and Permitted Purposes	27
17.4 Sharing, Human Review, and Participant Rights	28
18 Quality Assurance Subsystems	28
18.1 Response Verification	28
18.2 Transcription and Transcript Validation	29
18.3 Dataset Curation and Export	30
18.4 Anonymous In-Product Feedback	31
19 Pilot Deployment and Dataset Statistics	32
20 Impact and Reuse Potential	33
20.1 Limitations	33
21 Observability and Testing	33
21.1 Activity Logs	34
21.2 Realtime Presence	34
21.3 In-App Test Runner	34
21.4 Automated Test Strategy	35
21.5 Operational Metrics and Alerts	35
22 Documentation Maintenance and Release Checklist	36
22.1 Documentation Source Layout	36
22.2 Interface Documentation Evidence	36
22.3 Release Checks	36
23 System Workflows and Behaviour	37
23.1 Worked Example: One Recording from Capture to Export	38

24 User Interface and Experience Design	38
25 Future Extensions	38
26 Conclusion	39
27 Project Attribution and Declarations	39
28 Acknowledgement	39
A Architecture Decision Records	39
A.1 ADR-001: Adopt Supabase as the Backend-as-a-Service Platform	39
A.2 ADR-002: Store Privileged Roles Separately from User Profiles	40
A.3 ADR-003: Model Transcriber Access as an Approved Profile Capability	40
A.4 ADR-004: Use a Configurable Pessimistic Transcription Lease	40
A.5 ADR-005: Capture Browser Audio as WebM/Opus	40
A.6 ADR-006: Use Bounded LocalStorage for Interrupted-Recording Recovery	41
A.7 ADR-007: Retire Prompts after a Configured Unique-Response Threshold	41
A.8 ADR-008: Minimise Identity Linkage in the Feedback Schema	41
A.9 ADR-009: Restrict Dataset Export to Administrators	42
A.10 ADR-010: Support Structured Spontaneous and Read-Speech Prompts	42
B Glossary	42

Start Here: Reading Paths

This handbook is layered so that readers do not need to read it from beginning to end. Use the route below that matches the work you need to perform. Detailed deployment, backup, incident-response, and troubleshooting procedures are maintained separately in `operations-runbook.pdf` so this guide can remain focused on architecture and development.

Table 1: Recommended reading paths by audience.

Audience	Begin with	Continue with
New contributor	Document control and the developer quick start	Architecture, implementation approach, database design, testing, and the release checklist
Frontend developer	System scope and architecture	Roles, recording pipeline, offline recovery, quality-assurance interfaces, and UI/UX design
Database or backend engineer	Architecture and database design	Domain invariants, concurrency, backend interfaces, storage, RLS, and testing
Security or privacy reviewer	Roles and authorization	Domain invariants, storage, security controls, quality assurance, pilot governance, and the ADR appendix
Researcher or data curator	Objectives and related work	Recording and transcription rules, quality assurance, pilot statistics, impact, and limitations
Administrator or operator	Roles and quality assurance	Open <code>operations-runbook.pdf</code> for deployment, backup, incident recovery, and troubleshooting
Maintainer preparing a release	Developer quick start	Testing, documentation maintenance, ADRs, and the separate operations runbook

Source hierarchy. Version-controlled migrations, generated types, configuration, and tested application behavior remain authoritative for implementation. This guide explains their architecture and development contract; the operations runbook explains how to deploy, recover, and support them; and Section 19 is the single authoritative narrative for pilot methodology and reported statistics.

1. Document Control and How to Use This Guide

Table 2: Document control information.

Item	Value
Document version	1.3
Last updated	7 September 2026
Software release covered	v1.0
Primary maintainers	Braxton Mandara, DSAIL
Canonical implementation source	JasinaHub GitHub repository , including its lockfile, database migrations, generated database types, and deployment configuration
Purpose	Architecture description, developer onboarding guide, implementation reference, security baseline, and research-workflow record
Companion operations document	operations-runbook.pdf for deployment, backup, incident recovery, and troubleshooting

This guide is intended for frontend developers, database engineers, security reviewers, researchers, and maintainers extending JasinaHub. Architecture sections explain why the system is structured as it is; developer sections explain how to obtain, configure, run, test, and release it. Deployment maintainers and system administrators should use the reading path on the Start Here page and the companion [operations-runbook.pdf](#).

1.1. Source-of-Truth Policy

Executable artifacts take precedence over prose. Exact dependency versions come from the package lockfile; database definitions and RLS policies come from version-controlled SQL migrations; generated TypeScript database types reflect the applied schema; and deployed secrets remain in the environment's secret manager. When this document disagrees with one of those artifacts, the discrepancy must be treated as a documentation defect and corrected in the same change that modifies the implementation.

The application source is maintained in the external repository linked in Table 2; it is not duplicated inside this documentation workspace. The commands in the reproduction sections must therefore be validated against a clean checkout of release v1.0. Exact dependency versions, script names, environment variables, migrations, and deployment settings remain governed by that checkout and must be checked using Section 22 before publishing a revised handbook.

1.2. Publication Alignment Policy

The companion JasinaHub publication and this handbook describe the same v1.0 pilot and must use the same participant counts, collection dates, workflow rules, quality criteria, completion totals, privacy controls, and software boundary. Repository and database artifacts remain authoritative for implementation behavior; approved pilot records remain authoritative for reported study statistics. When either document changes a shared factual claim, maintainers must review and update the other in the same release process. Publication compression must not remove qualifications that affect implementation or governance, including the distinction between pseudonymity and anonymity and the separation of *JasinaH* model development from JasinaHub.

1.3. Privacy-Policy Alignment

The privacy requirements in this handbook incorporate the policy text supplied for the 7 September 2026 documentation revision. This does not establish that the text has been approved or published at <https://jasinahub.vercel.app/privacy-policy>. In particular, its retention schedule is explicitly a draft sample. Before release, reconcile the policy page, consent forms, implementation, and operational procedures, and record the approved policy version and effective date.

Executable artifacts establish implementation behaviour; they do not override participant consent or privacy obligations. Additional or more restrictive project-specific consent conditions also apply. Section 17.2 maps the supplied policy to the collection workflow, and the companion runbook covers requests and withdrawal.

1.4. Code Metadata

Table 3: JasinaHub code and support metadata.

Item	Value
Current code version	v1.0
Permanent repository link	https://github.com/DeKUT-DSAIL/Jasinahub
Legal code licence	MIT
Version-control system	Git
Primary code languages	TypeScript, SQL, HTML, and CSS
Compilation baseline	Node.js $\geq$ 18, React 18, Vite 5, and the Supabase JavaScript SDK
Developer documentation	https://github.com/DeKUT-DSAIL/Jasinahub/blob/master/developer_documentation.pdf
Participant privacy policy	https://jasinahub.vercel.app/privacy-policy
Support email	dsail-info@dkut.ac.ke

1.5. Terminology and Requirement Language

The words **must**, **should**, and **may** indicate a mandatory requirement, a recommended practice, and an optional capability, respectively. A *volunteer* contributes recordings; an *approved transcriber* is a volunteer granted the transcription capability; and an *administrator* holds an explicit privileged role. Verification, consent, guideline acceptance, and transcription approval are account states rather than interchangeable roles.

Documented behaviour describes the release as reported in the project documentation and companion publication. **Required controls** state conditions that a deployment must satisfy; they do not, by themselves, establish that those controls are implemented. **Verification status** identifies the supporting evidence or an unresolved check. The application checkout, deployed configuration, and test results are not included in this workspace, so this handbook’s behavioural descriptions are not an independent implementation audit. Release maintainers must link verified claims to the relevant source commit and test evidence using Section 22.

2. Introduction

More than 2,000 indigenous languages are spoken across Africa [14], yet most remain underrepresented in contemporary speech technology [15]. Scarce labelled data, linguistic and acoustic variability, constrained computing resources, and governance challenges continue to limit ASR development [20]. JasinaHub addresses the need for configurable infrastructure that can collect both read speech and naturally elicited, domain-specific responses.

The platform brings volunteers, approved transcribers, and administrators into one corpus-creation workflow. Volunteers contribute recordings, transcribers produce verbatim text, and administrators review the material before export. A mobile-responsive web interface supports participation, while the backend coordinates access, task assignment, and record storage.

This guide explains the architecture and development contracts behind that workflow. Section 3 defines the software boundary; Section 19 reports the operational pilot and its limits. Deployment and recovery procedures are in the companion `operations-runbook.pdf`.

3. System Scope and Assumptions

JasinaHub targets community-driven speech and transcription-data collection in low-resource environments. Contributors use modern microphone-capable browsers, and the platform can be installed as a Progressive Web Application for an app-like mobile or desktop experience.

The software covers recording, transcription, administrative review, and dataset export. Model training, evaluation, and inference deployment are outside its release boundary. The separate *JasinaH Kikuyu* ASR project uses its exported corpus.

Prompt configuration supports naturally elicited responses and predetermined read-speech sentences. Categories and prompt inventories can accommodate additional languages and domains without changing the core workflow, provided their metadata and interaction needs fit the existing model. Language-specific

consent, orthography, transcription guidance, and controlled vocabularies still require review. The Kikuyu deployment and study population are described in Section 19.

Operational assumptions include:

- Under the supplied privacy policy, participation is voluntary and intended for adults aged 18 or older. Contributors may skip prompts, stop recording, or leave without submitting. The workflow does not collect clinical records.
- Intermittent network connectivity is common, requiring cached resources, clear network-state feedback, and interrupted-upload recovery.
- Contributors may use low-cost mobile devices with limited storage and processing capacity.
- Audio quality varies and must be validated through a moderation pipeline.
- Transcribers are a smaller, vetted subset of contributors and must be approved by an administrator before performing transcription work.
- Authentication uses email and password rather than third-party identity providers. Contributors therefore need access to an email account.

4. Developer Quick Start and Reproducibility

This section defines the minimum procedure by which a developer should be able to reproduce JasinaHub from a clean checkout. The application repository must expose the scripts and files named here, or its release documentation must record the exact equivalents. Do not infer production credentials from examples and never commit a populated environment file.

4.1. Prerequisites

The supported versions must be pinned in repository-controlled files such as `.nvmrc`, `.tool-versions`, `package.json`, the package lockfile, and the Supabase configuration. At minimum, a workstation requires:

- Git for source control.
- Node.js 18 or later, with the exact supported runtime selected by the repository and the package manager selected by its lockfile.
- Supabase CLI for local PostgreSQL, authentication, storage, Realtime, and Edge Function emulation.
- Docker or another container runtime supported by the Supabase CLI.
- A Chromium- or Firefox-based browser with microphone access for recording tests.
- Optional Deno tooling when developing or testing Edge Functions outside the Supabase CLI.

The release maintainer must verify exact versions before publication. A reproducible installation must not rely on an unspecified “latest” tool version.

4.2. Obtain and Install the Application

The following commands use the canonical public repository and the documented v1.0 release. If the repository does not expose a matching Git tag, use the release commit recorded in the repository and update this guide.

Listing 1: Reference clean-checkout installation workflow.

```
1 git clone https://github.com/DeKUT-DSAIL/Jasinahub.git jasinahub
2 cd jasinahub
3 git checkout v1.0
4
5 # Confirm the runtime selected by the repository.
6 node --version
7
8 # Use the command matching the committed lockfile.
9 npm ci
```

If the repository contains `pnpm-lock.yaml` or `yarn.lock` instead of `package-lock.json`, maintainers must publish the equivalent frozen-lockfile command. Only one package-manager lockfile should be authoritative.

4.3. Required Repository Contract

A release intended to be reproducible should contain the following artifacts or clearly documented equivalents:

Table 4: Required application-repository artifacts.

Artifact	Responsibility
<code>package.json</code> and one lockfile	Dependency versions and standard development, build, lint, type-check, and test scripts.
<code>.env.example</code>	Names and descriptions of client-safe configuration values, with no secrets.
<code>supabase/config.toml</code>	Local Supabase service configuration.
<code>supabase/migrations/</code>	Ordered, immutable SQL migrations for schema, functions, triggers, grants, indexes, and RLS.
<code>supabase/seed.sql</code>	Non-sensitive local fixtures or a documented seed command.
<code>supabase/functions/</code>	Version-controlled Edge Function sources and shared modules.
<code>src/</code>	Frontend routes, components, hooks, services, generated types, and offline persistence logic.
<code>tests/</code> or colocated test files	Unit, integration, RLS, concurrency, accessibility, and end-to-end tests.
<code>README.md</code>	Short entry point linking to this complete guide.
<code>CHANGELOG.md</code>	User-visible and developer-visible release changes.

4.4. Environment Configuration

Copy the template rather than creating configuration from memory:

Listing 2: Creating local configuration.

```
1 cp .env.example .env.local
```

The exact variable names must match the application. A Supabase/Vite deployment normally requires the categories shown in Table 5. Client variables are embedded into the browser bundle and must never contain service-role keys, database passwords, SMTP credentials, cloud-provider private keys, or other privileged secrets.

Table 5: Environment-variable reference contract.

Variable or category	Scope	Purpose and handling
<code>VITE_SUPABASE_URL</code>	Browser	Public project API URL; local value is printed by <code>supabase status</code> .
<code>VITE_SUPABASE_ANON_KEY</code>	Browser	Public anonymous key. RLS and storage policies must make its exposure safe.
Application base URL	Browser/server	Canonical origin used for authentication redirects and generated links.
Service-role key	Server only	Privileged maintenance operations; store only in a secret manager or protected CI environment.
SMTP credentials	Server only	Authentication email and password-reset delivery, if custom SMTP is configured.
Google Drive/GCS credentials	Edge Function only	Legacy migration operations; provision with minimum privileges and remove when migration is complete.
Telemetry configuration	Browser/server	Environment name, release identifier, and optional error-reporting destination.

Every entry in `.env.example` must include whether it is required, its scope, an example non-secret value, and the feature disabled when it is absent.

4.5. Start the Local Backend

Listing 3: Reference local Supabase initialization.

```
1 npx supabase start
2 npx supabase db reset
3 npx supabase status
```

The reset command must apply every migration in order and then load deterministic development fixtures. It is destructive to the local database and must never be directed at staging or production. Record the displayed local API URL and anonymous key in `.env.local`. Edge Functions required during development should be served using the repository’s documented command, conventionally `npx supabase functions serve`.

4.6. Run and Verify the Frontend

Listing 4: Reference frontend verification workflow.

```
1 npm run typecheck
2 npm run lint
3 npm test
4 npm run dev
```

The browser smoke test should confirm that the login page loads, an unverified account cannot access prompts, a verified test volunteer can reach the recorder, and an approved test transcriber can claim a seeded recording. Microphone tests require a secure context; `localhost` is treated as secure by modern browsers.

4.7. Bootstrap Test Accounts and the First Administrator

Seed data should create disposable local identities for each permission class. Production administrator creation must be a controlled, audited operation performed by an existing privileged operator or a one-time bootstrap migration. It must never be exposed as a public signup option. After bootstrapping, verify the administrator’s `user_roles` row and revoke or remove any temporary elevation mechanism.

4.8. Clean-Checkout Acceptance Criteria

A release is reproducible only when a developer can perform the following on a clean machine without undocumented intervention:

1. Install dependencies from the committed lockfile.
2. Start the local backend and apply all migrations from an empty database.
3. Load non-sensitive seed data.
4. Generate or validate database types.
5. Pass type-checking, linting, unit tests, database/RLS tests, and critical end-to-end tests.
6. Build the production frontend without warnings treated as errors.
7. Complete one recording, moderation, transcription, validation, and export cycle using test data.

5. System Objectives and Design Principles

The following objectives guide development; they are not measurements of achieved performance. The software boundary is defined in Section 3, and the pilot’s evidence and limitations are reported in Section 19.

5.1. Primary Objectives

- Support community-based collection of speech and corresponding text through a web interface.
- Support complementary collection modes: naturally elicited responses to structured prompts and read speech from predetermined sentences.
- Prevent conflicting transcription assignments and preserve attribution. Support transcription accuracy through guidelines and audio-text review, separately from concurrency controls.
- Support reliable operation under intermittent connectivity through cached application resources, network-state feedback, bounded client-side persistence, and interrupted-upload recovery without claiming fully offline operation.
- Broaden prompt coverage by retiring prompts at a configurable unique-respondent threshold; measure speaker diversity separately.
- Preserve transcription attribution and editorial integrity by binding edit rights to the original transcriber.
- Provide administrators with end-to-end observability and an in-app verification suite covering RLS, RPCs, storage, and business invariants.
- Produce structured, machine-ready datasets with pseudonymous contributor and transcriber provenance where permitted, suitable for downstream ASR research without exporting respondent names.

5.2. Architectural Principles

- **Separation of concerns:** Independent evolution of user interfaces, backend services, and persistence layers.
- **Backend-enforced correctness:** The client may provide early validation and user guidance, but authorization, concurrency, and integrity-sensitive invariants must also be enforced authoritatively in PostgreSQL through constraints, Row-Level Security policies, triggers, and narrowly scoped transactional RPCs.
- **Offline resilience:** Preserve recoverable work through connectivity interruptions and make recovery limits visible. Section 16 defines the persistence mechanism and failure handling.
- **Privacy by data minimisation:** Limit identity linkage to what the workflow requires. Section 18 explains feedback privacy and export safeguards.
- **Observability as a first-class feature:** Activity logs, realtime presence, and an in-app diagnostic test runner ship as part of the administrative interface.
- **Extensibility:** Reuse the collection pipeline when adding prompt inventories, subject to the compatibility limits in Section 3.

6. Related Work and Literature Review

The development of speech technologies for under-resourced African languages remains constrained by limited language coverage, insufficient labelled data, acoustic variability, infrastructure limitations, and ethical and governance concerns [15, 20]. Mozilla Common Voice demonstrates the scale attainable through browser-based crowdsourcing and community validation of prompted read speech [4]. JasinaHub supports read speech while also allowing speakers to answer structured prompts in their own words.

Participatory initiatives such as Masakhane show how communities can shape African-language technology development, although the cited work primarily concerns machine translation rather than an end-to-end speech platform [23]. The Lwazi Project [8] demonstrated multilingual speech resources for South African languages, while GlobalPhone [28] supplied controlled multilingual speech. These efforts establish the value of African speech resources but do not describe JasinaHub’s integrated workflow for configurable prompts, interrupted-upload recovery, atomic candidate reservation, moderation, transcription, and governed export.

Recent East African work demonstrates the downstream value of locally curated resources. Sunflower combined digital corpora, digitised documents, community materials, professional translations, and transcribed radio audio for Ugandan language models, reporting the strongest average bidirectional translation performance in 24 of 31 evaluated languages [2]. KenCorpus combined 5.6 million words with 177 hours of speech for Swahili, Dholuo, and Luhya, alongside annotated part-of-speech data, translation pairs, and proof-of-concept systems [32]. AfriVoices-KE broadened Kenyan coverage with approximately 3,000 hours of scripted and spontaneous speech from 4,777 speakers across five languages [33]. Its Kikuyu subset is reported as 754 hours from 1,331 contributors. A Kikuyu and Kinyarwanda ASR scaling study further showed that reference-transcription noise accounted for 38.6% of high-error cases in the analysed Kikuyu data [3]. These projects differ from JasinaHub in scale and purpose, but reinforce the importance of speaker diversity, annotation quality, and reusable collection infrastructure.

Mobile field tools offer a closer operational comparison. Aikuma supports recording, respeaking, and oral translation for language documentation [11]; LIG-Aikuma extends mobile collection with translation and elicitation modes [17]; and ALFFA demonstrates how collected Sub-Saharan African speech can support ASR development [16]. The Makerere Radio Speech Corpus released 155 hours of Luganda radio speech with baseline ASR results [22]. MasakhaNER concerns a different NLP task, but its collaborative annotation and agreement procedures are relevant to JasinaHub’s planned measurement of transcription consistency [1].

Table 6: Comparison with related speech-collection platforms and workflows. A dash denotes a capability not reported in the cited publication, not evidence that it is absent from later software.

Platform	Collection and access	Annotation and quality control	Resilience, governance, and export
Common Voice [4]	Prompted read speech through browser crowdsourcing	Community recording validation	Public multilingual corpus releases; other compared capabilities not reported
Aikuma [11]	Recording, respeaking, and oral translation through an Android app	Collaborative, time-aligned oral interpretation	Compared resilience and export capabilities not reported
LIG-Aikuma [17]	Recording, respeaking, translation, and elicitation on mobile devices	Parallel, time-aligned speech collection	Compared resilience and export capabilities not reported
AfriVoices-KE [33]	Scripted and spontaneous speech through a customised mobile app	Automated and human quality checks	Curated multilingual dataset; compared concurrency controls not reported
JasinaHub	Structured spontaneous and read speech through a responsive PWA	Approved transcription, atomic candidate locking, and administrator review	Interrupted-upload recovery, consent-based collection, and pseudonymised controlled export

The systems differ in how responsibility and governance are organised. Common Voice distributes validation across contributors; Aikuma and LIG-Aikuma emphasise collaborative documentation; and AfriVoices-KE combines automated and human checks at dataset scale. JasinaHub integrates an approved-transcriber queue, database-level random candidate locking, editable submissions, administrator acceptance or rejection, withdrawal procedures, and administrator-only pseudonymised export. These controls reduce direct identity exposure but do not make speech anonymous [30].

Large multilingual resources such as MLS and FLEURS broaden training and evaluation coverage but are primarily based on read or prompted speech [12, 24]. Self-supervised and weakly supervised systems including wav2vec 2.0, XLS-R, and Whisper reduce dependence on labelled corpora, but supervised adaptation and evaluation still require curated target-language speech–transcription pairs [6, 7, 25].

JasinaHub therefore complements existing resources rather than seeking to replace them. Its contribution is the combination of mobile-responsive collection of structured spontaneous and read speech, interrupted-upload recovery, concurrent transcription locking, administrative quality review, and pseudonymised export in one configurable web platform. The Kikuyu pilot validates this integrated workflow rather than claiming to be the largest Kenyan speech corpus.

7. System Architecture and Design

JasinaHub separates browser interaction from backend services and persistent storage. The client presents tasks and captures input; backend services authenticate requests and coordinate access to database records and audio objects.

Four logical layers organise these responsibilities: a Progressive Web Application (PWA), Backend-as-a-Service (BaaS) services, PostgreSQL domain logic, and managed cloud storage. These are logical boundaries rather than four independently deployed systems: Supabase provides several of the services. The managed-compute model follows serverless architecture principles [19]. It reduces the need to provision application servers, but database policies, transactional controls, and deployment configuration still require review and testing.

7.1. Technology Stack

- **Frontend:** React 18, Vite, TypeScript, Tailwind CSS, shadcn/ui component library, React Query (server state), React Router (navigation), `next-themes` (dark/light mode).
- **Backend (Supabase):** PostgreSQL 15+, GoTrue authentication service, Object Storage, Realtime channels, and Deno-based Edge Functions.
- **Client APIs:** `MediaRecorder` (WebM/Opus audio capture), `LocalStorage` through `recordingStorage` (release recovery persistence), `Service Worker` (PWA shell), and `Web Vibration API` (haptics). `IndexedDB` is a recommended future replacement for larger binary queues.

7.2. High-Level Architecture

The frontend presents prompts, records audio, checks user input, and shows upload and task status. It also retains a recovery copy before upload. The backend must check who is making each request, which records they may access, and whether the requested change is allowed. These checks remain necessary even when the browser has already validated the input. Section 16 explains which tasks can recover from a lost connection; Section 23.1 follows one recording through the workflow.

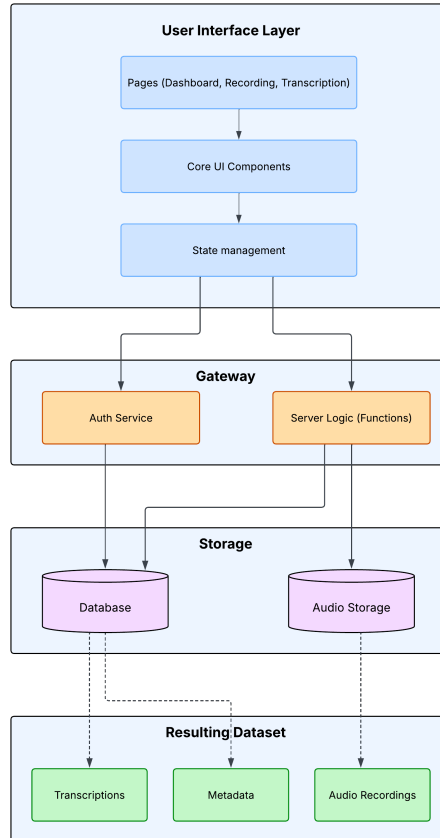


Figure 1: System architecture of JasinaHub illustrating how different elements interact with each other

7.3. Behavioural Model

JasinaHub represents submitted recordings, transcription claims, and review outcomes as workflow states. Backend transitions govern persisted records; recording and recovery also involve local browser state before submission. Figure 2 shows the end-to-end workflow. Permitted transitions and their failure paths are specified in the domain-rule and concurrency chapters.

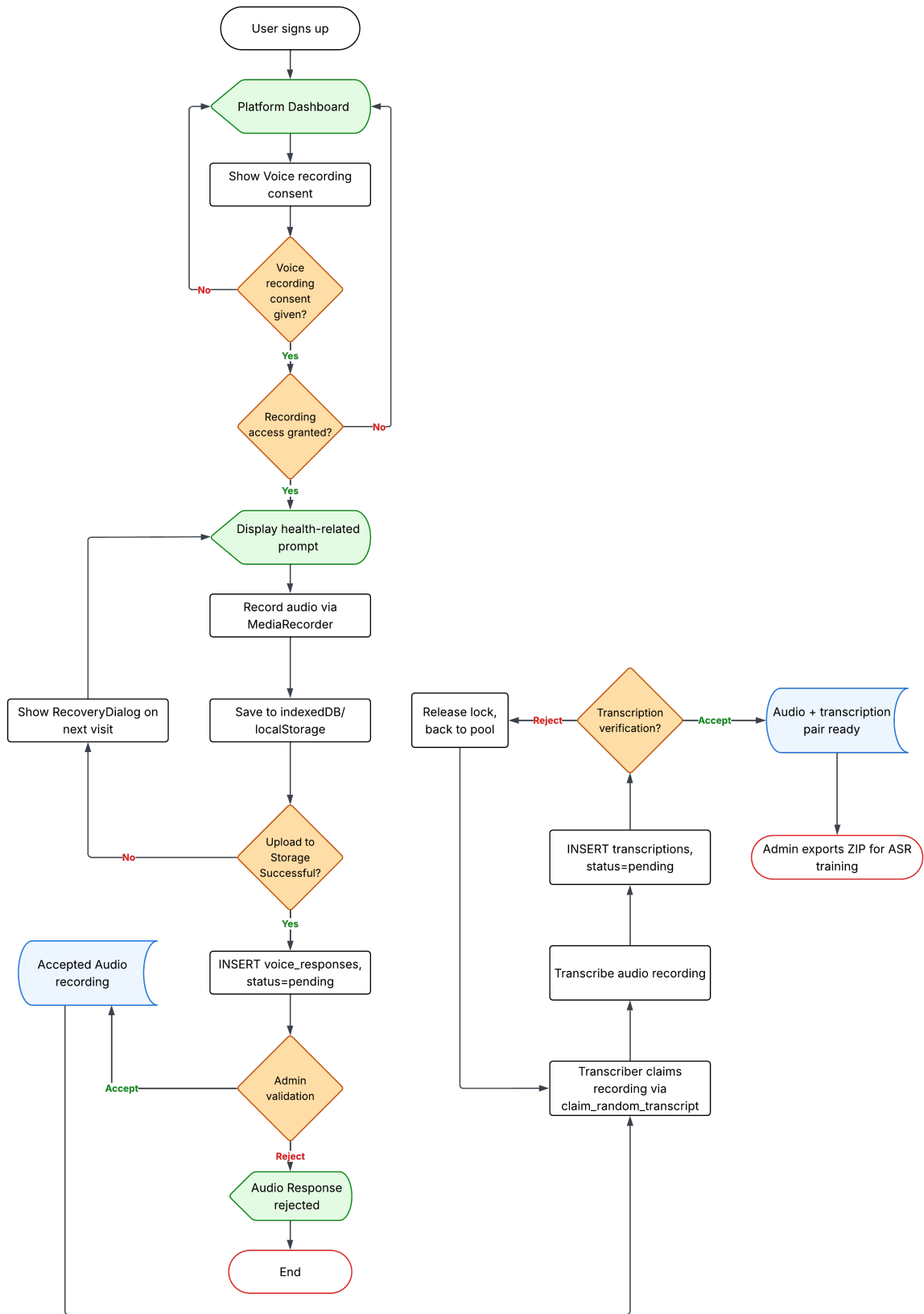


Figure 2: End-to-end system flowchart showing speech data collection, moderation, transcription, verification, and dataset export workflows.

8. System Roles and Authorisation Model

JasinaHub stores the PostgreSQL `app_role` values `admin` and `user` in a dedicated `user_roles` table—never on the editable `profiles` table—to reduce profile-update privilege-escalation risk. This separation follows established role-based access-control principles [27]. Role checks are performed by a `has_role(uuid, app_role)` function declared as `SECURITY DEFINER` so that RLS policies can invoke it without recursive policy evaluation. In application terms, the `transcription_approved` profile flag yields three effective operating groups: volunteer, approved transcriber, and administrator.

8.1. Volunteer (Recording Contributor)

The default role assigned at signup. Volunteers must:

- Complete a four-step signup wizard capturing first name, last name, email, password (with strength indicator), phone number, gender, age, and dialect.
- Be marked `verified = true` by an administrator before accessing collection prompts (enforced by RLS on `questions` and `voice_responses`).
- Grant explicit, persistent voice-recording consent (stored as `voice_recording_consent` and `consent_timestamp` on the profile) before recording.
- Optionally update their phone number exactly once via the one-time `phone_number_updated` flag.

8.2. Transcriber

A volunteer who additionally requests transcription work. Transcribers are gated by the boolean `transcription_approved` flag on the profile, which only an administrator may set. They must accept verbatim transcription guidelines (recorded as `transcription_guidelines_agreed` and `transcription_guidelines_agreed_at`) before claiming any task.

8.3. Administrator

A role assigned by inserting an `admin` entry in `user_roles`. Administrators have full visibility through the `has_role(auth.uid(), 'admin')` predicate that appears in nearly every RLS policy. They manage prompts, verify users, approve transcribers, moderate responses, validate transcriptions, export datasets, and access observability tooling.

8.4. Authorization Matrix

Table 7 separates stored roles from account states. Every cell must be backed by an RLS policy, storage policy, or controlled function test; hiding a button is not authorization.

Table 7: Minimum authorization matrix.

Operation	Anonymous	Unverified user	Verified volunteer / approved transcriber	Administrator
Read active prompts	No	No	Yes	Yes
Insert own recording	No	No	Yes	Policy-defined
Read own recording	No	No	Yes	Yes
Moderate recordings	No	No	No	Yes
Claim transcription	No	No	Approved transcriber only	Policy-defined
Edit own unaccepted transcription	No	No	Original transcriber only	Yes
Validate transcription	No	No	No	Yes
Manage roles and approvals	No	No	No	Yes
Read feedback aggregates	No	No	No	Yes
Export accepted dataset	No	No	No	Yes

Policy-defined administrator behavior must be stated explicitly in the implementation reference: an administrator should not implicitly submit research data merely because the account can bypass ordinary restrictions. Revoking verification or transcription approval must define what happens to in-progress uploads, existing data, and active locks.



Figure 3: Use case diagram showing the three actors (Volunteer, Transcriber, Administrator) and the fourteen primary use cases of the JasinaHub system.

9. Development Process and Implementation Approach

The development of JasinaHub followed an AI-assisted, iterative prototyping-to-production process. This reflects contemporary software-engineering practice in which AI programming assistants support activities such as code generation, search, and technical consultation, while their output remains subject to engineering review [13]. Early work emphasised rapid validation of user workflows; later phases focused on correctness, security, resilience, and operational pilot use.

- **Phase 1 — Minimum Viable Product (MVP):** The initial application was scaffolded using [Lovable](#), enabling rapid validation of authentication, prompt navigation, audio recording, submission, transcription, and basic administration with researchers and volunteers.
- **Phase 2 — Engineering Refinement:** The MVP was refined using [Google Antigravity](#) and source review. Work covered stricter TypeScript typing, modular React components, database schemas, transactional candidate reservation, interrupted-upload recovery, PWA installation, and an embedded test runner. These changes addressed concurrent transcription and intermittent mobile connectivity.
- **Phase 3 — Quality Assurance and Observability:** Addition of dataset curation tooling (HuggingFace-style table view and governed ZIP/CSV export), administrative verification dashboards, anonymous in-product feedback, activity logs, realtime presence tracking, and storage migration utilities for legacy Drive-hosted recordings.

AI-assisted development tools accelerated prototyping and refactoring but did not replace developer oversight. A developer remained involved throughout design and implementation to verify that generated components followed the intended application logic, test the resulting workflows, and review security-sensitive changes. PostgreSQL RPC functions, Row-Level Security policies, schema migrations, and database configuration were manually reviewed and tested before deployment. Generated changes remain subject to source control, human review, type checking, RLS and concurrency testing, migration review, and the release checklist in Section 22.

9.1. Scalability Precautions

The architecture places authorization and contested task assignment in the database, while managed Edge Functions handle specialised server-side work. These choices reduce reliance on client-side coordination; they do not establish a tested capacity limit. Release testing must measure the workload the deployment is expected to support.

Reuse across languages depends on more than adding prompts. New metadata, script handling, moderation rules, or consent obligations may require implementation changes, as discussed in Section 3.

9.2. Expected Source-Code Organisation

The exact tree is defined by the application repository. The following separation is recommended so that ownership is visible and database behavior is not hidden inside page components:

Listing 5: Recommended application repository layout.

```

1 jasinahub/
2 |-- src/
3 |   |-- app/ or routes/      # route definitions and page composition
4 |   |-- components/         # reusable presentation components
5 |   |-- features/           # recording, transcription, admin, feedback
6 |   |-- hooks/              # query, mutation, auth, presence hooks
7 |   |-- lib/                # Supabase client and shared utilities
8 |   |-- offline/            # recordingStorage, recovery, queue/state logic
9 |   |-- types/              # generated database and domain types
10 |-- supabase/
11 |   |-- migrations/         # immutable schema and policy history
12 |   |-- functions/          # Deno Edge Functions
13 |   |-- tests/              # SQL, RLS, and RPC tests
14 |   |-- seed.sql            # deterministic local fixtures
15 |-- tests/
16 |   |-- integration/
17 |   |-- e2e/
18 |-- public/                 # PWA manifest and static assets
19 |-- .env.example
20 |-- package.json
21 |-- package-lock.json       # or the single chosen lockfile

```

Feature modules should expose a small public interface. Page components should not issue ad hoc privileged queries or duplicate RLS-sensitive eligibility logic. Database types should be generated after schema changes and committed or verified in continuous integration according to the repository policy.

9.3. Feature-to-Implementation Traceability

Each implementation release should maintain a traceability table using actual paths. Table 8 defines the minimum coverage.

Table 8: Feature-to-implementation traceability template.

Feature	Client owner	Backend owner	Required tests
Authentication/onboarding	Auth routes and session provider	Auth configuration, profile trigger, profile RLS	Signup, administrator verification, privilege tests
Recording	Recorder feature and offline queue	Response constraints, storage policy	Codec, retry, duplicate, duration tests
Moderation	Admin response-review feature	Status-transition RPC/policies	Permission and transition tests
Transcription	Claim and editor feature	Claim/release RPCs, locks, transcription constraints	Concurrency, expiry, ownership tests
Feedback	Feedback form and analytics	Feedback tables and policies	Validation, privacy, aggregation tests
Export	Dataset administration feature	Export query/function and storage access	Schema, authorization, checksum tests

9.4. Contribution Workflow

Changes should be made on a short-lived branch and submitted for review. A change that modifies schema, authorization, a workflow state, an environment variable, or a public data contract must update its migration, generated types, automated tests, this guide, and the changelog in the same pull request. Reviewers should require passing formatting, linting, type checking, tests, and production build checks. Database migrations already applied to shared environments must not be edited in place; corrections require a new forward migration.

10. Database Design

The release v1.0 public data model is implemented as a normalised relational schema in PostgreSQL. Eleven tables support identity, prompt management, recording, transcription, feedback, progress, and auditability; the **voice-recordings** object-storage bucket holds WebM/Opus audio. Each core entity is represented explicitly, with foreign-key relationships enforcing referential integrity and Row-Level Security policies enforcing per-row authorisation [29].

10.1. Schema Overview

Table 9: Tables comprising the JasinaHub schema.

Table	Purpose	Representative fields
<code>profiles</code>	Extended user metadata associated with authenticated accounts.	Names, phone, dialect, gender, age, verification, transcription approval, consent, guideline acceptance
<code>categories</code>	Thematic groupings for domain-specific questions or read-aloud sentences.	<code>name</code> , <code>description</code>
<code>questions</code>	Ordered prompts within categories.	<code>question_text</code> , <code>category_id</code> , <code>order_index</code> , <code>image_url</code>
<code>voice_responses</code>	Audio submissions and moderation state.	<code>audio_file_url</code> , <code>duration_seconds</code> , <code>status</code> , <code>question_id</code> , <code>user_id</code>
<code>transcriptions</code>	Verbatim text linked to accepted recordings.	<code>transcription_text</code> , <code>voice_response_id</code> , <code>user_id</code> , <code>edit_count</code> , <code>status</code>
<code>transcription_locks</code>	Atomic candidate reservation with a configurable lease (default 10 minutes in the current deployment and pilot).	<code>voice_response_id</code> , <code>user_id</code> , <code>locked_at</code> , <code>expires_at</code>
<code>user_progress</code>	Per-user, per-category completion tracking.	<code>category_id</code> , <code>user_id</code> , completed and total questions
<code>user_roles</code>	Role assignments separate from editable profiles.	<code>user_id</code> , role using the <code>app_role</code> enum
<code>user_activity_logs</code>	Audit trail of platform interactions.	<code>user_id</code> , <code>action</code> , <code>page</code> , details JSONB
<code>feedback_questions</code>	Administrator-configured feedback prompts.	<code>question_text</code> , <code>question_type</code> , required/active flags, order
<code>feedback_submissions</code>	Anonymous answer rows without a direct user identifier.	<code>question_id</code> , <code>answer</code> , <code>submitted_at</code>

Object storage: `voice-recordings` stores uploaded WebM/Opus audio blobs.

10.2. Schema Reference Requirements

The overview in Table 9 is navigational rather than authoritative. For every release, the application repository must generate or maintain a schema reference containing every column’s SQL type, nullability, default, primary and foreign keys, `ON DELETE` behavior, unique and check constraints, indexes, triggers, grants, RLS enablement, and migration of origin. The SQL migrations remain authoritative.

The following invariants are required by the behavior described in this guide and must be visible in executable database definitions:

- A profile UUID corresponds to an authentication user UUID.
- A contributor has at most one voice response for a given question.
- Each prompt’s collection mode (structured spontaneous or read speech) is explicit or deterministically derivable from governed configuration and is preserved in exports.
- A voice response has at most one canonical transcription unless versioned alternatives are introduced deliberately.
- A recording cannot be accepted, rejected, transcribed, or exported through an undefined status value.
- A lock uniquely protects a recording and, if only one concurrent task is allowed, uniquely constrains its holder.
- Foreign-key deletion behavior preserves auditability and does not silently orphan storage objects.
- Edit counts, configured recording durations, rating ranges, NPS ranges, and feedback text limits are enforced at the backend rather than only in TypeScript.
- The `app_role` enum contains the release roles `admin` and `user`; transcriber capability is controlled separately through `profiles.transcription_approved`.

Listing 6: Illustrative constraint pattern; adapt names to the canonical migrations.

```
1 alter table voice_responses
2   add constraint voice_responses_user_question_key
3   unique (user_id, question_id),
4   add constraint voice_responses_duration_check
5   check (duration_seconds > 0 and duration_seconds <= 35); -- pilot tolerance
6
7 alter table transcriptions
8   add constraint transcriptions_response_key unique (voice_response_id),
9   add constraint transcriptions_edit_count_check
10  check (edit_count between 0 and 3); -- pilot correction allowance
```

Listing 6 illustrates the pilot profile, not proof of the deployed schema or a universal limit. A deployment that changes duration or correction settings must update its authoritative constraint or controlled RPC, configuration record, tests, and documentation together.

10.3. Migration and Seed Policy

Each schema change must be represented by a timestamped forward migration. A migration should be transactional where PostgreSQL permits, safe to run once, reviewed for locks and table rewrites, and tested both against an empty database and a production-like snapshot. Seed data must use synthetic identities and recordings and must never contain copied production personal data. Before a destructive production migration, create and verify a recoverable backup, document the forward-recovery plan, and define the period during which rollback remains possible.

10.4. Entity-Relationship Diagram

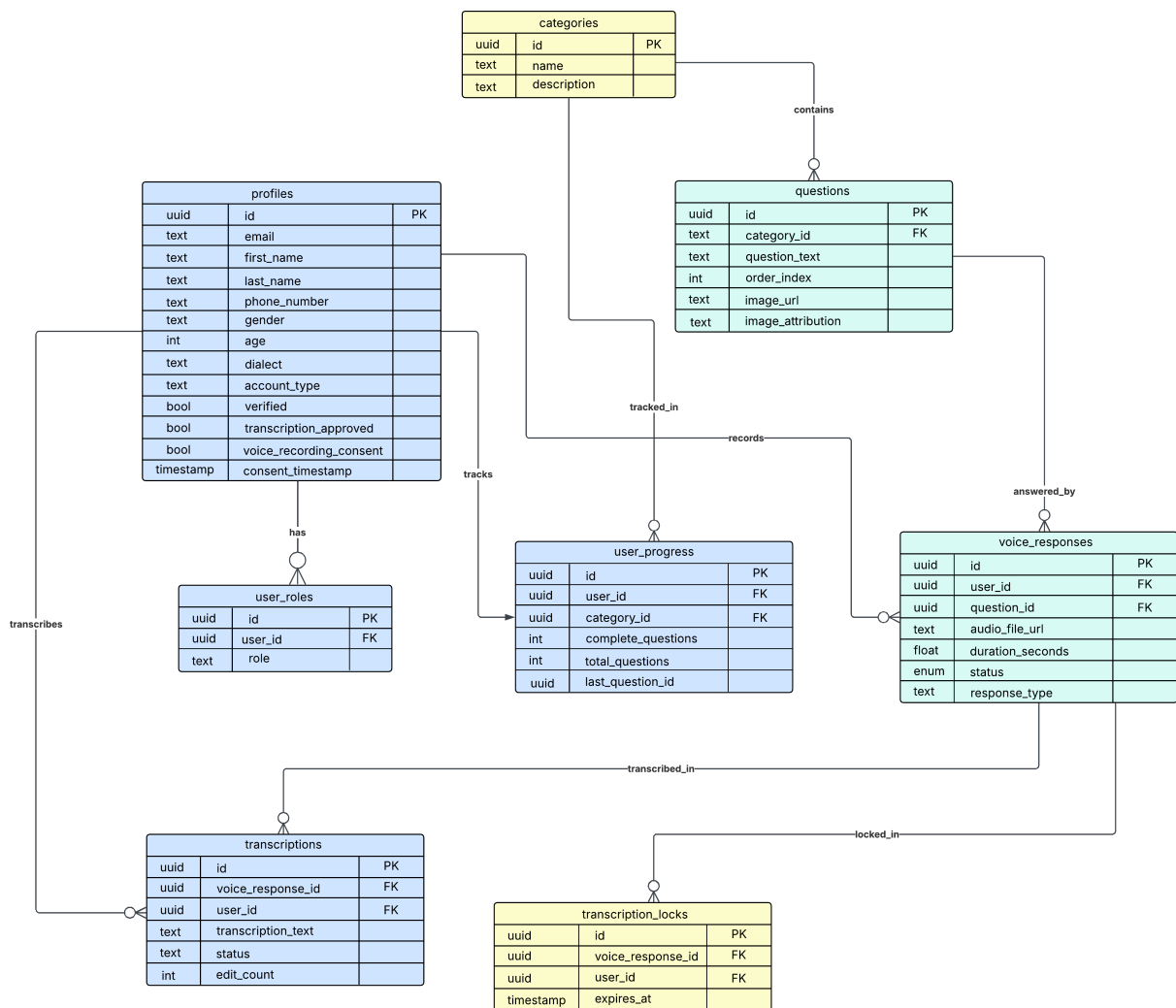


Figure 4: Entity-relationship diagram of the JasinaHub schema showing core entities, their attributes, and cardinality relationships. The **voice_responses** entity is the central hub linking volunteers, prompts, and transcriptions.

10.5. Row-Level Security Highlights

The release documentation describes the following access rules. They are a policy summary, not a verbatim account of the deployed SQL; migration and role-fixture tests must establish the actual grants and predicates.

- **profiles**: users may read/update only their own row; administrators may read/update any row.
- **user_roles**: users may read only their own role rows; only administrators may insert, update, or delete role assignments.
- **questions**: visible only to verified users or administrators; mutable only by administrators.
- **voice_responses**: insertable by verified users for their own **user_id**; readable by the owner, by any transcriber who has transcribed the response, and by administrators; status updates restricted to administrators.
- **transcriptions**: insertable by their owner; updatable by their owner only while **status** <> 'accepted' (and bounded by the edit-count invariant); fully manageable by administrators.

- **transcription_locks**: each user can manage only their own locks; administrators may manage all locks.
- **user_progress**: scoped to verified owners.
- **user_activity_logs**: insert-only for owners, read-only for administrators.
- **feedback_questions**: any authenticated user may read active questions; administrators have full CRUD.
- **feedback_submissions**: any authenticated user may insert an answer without a direct user identifier; only administrators may read aggregate or raw feedback according to policy.

10.6. RLS and Privileged-Function Review Checklist

RLS tests must exercise anonymous, ordinary authenticated, verified, approved-transcriber, administrator, and service-role contexts. For every table, test **SELECT**, **INSERT**, **UPDATE**, and **DELETE** separately, including attempts to change ownership columns. Storage buckets require equivalent object-level tests because table RLS does not protect object storage.

Every **SECURITY DEFINER** function must have a trusted owner, a fixed safe **search_path**, minimal grants, and explicit caller validation. User identity should be derived from **auth.uid()** inside security-sensitive functions rather than trusted from a caller-supplied **UUID**. Public **EXECUTE** privileges should be revoked unless intentionally required. Function tests must demonstrate that an ordinary user cannot act for another user or reach rows outside the documented permission matrix.

11. Domain Rules and Invariants

This chapter separates documented workflow behaviour from the controls required to protect it. The descriptions follow the release documentation; implementation and deployment checks remain subject to the verification policy in Section 1.5.

11.1. Configurable Recording-Duration Limit

Documented behaviour. The maximum recording duration is a deployment setting. The pilot used 30 seconds to align clips with the intended downstream model segment length [25]. The recorder provides feedback against that limit.

Required control. The backend must validate trusted duration metadata or inspect uploaded media and reject values beyond the configured tolerance. A database check on a caller-supplied number does not establish the actual media duration.

Verification status. The media-validation path has not been checked against an application checkout for this handbook. Release tests must cover the active limit, its tolerance, and attempts to submit misleading duration metadata.

11.2. One Recording per Contributor per Question

The documented uniqueness rule on **voice_responses(user_id, question_id)** prevents a contributor from answering the same prompt twice. It limits duplicate contributions to a prompt; it does not establish speaker diversity. The corresponding constraint must be confirmed in the release migrations and tested with concurrent duplicate submissions.

11.3. Question Retirement Threshold

Documented behaviour. Prompts leave active rotation when the configured number of distinct contributors has responded. The frontend uses question-count RPCs to determine availability. Section 19 records the pilot threshold and the historical exception introduced during collection.

Required control. The submission path must re-evaluate the threshold transactionally before accepting a response. A count displayed by the client cannot prevent concurrent or modified clients from exceeding the limit. The threshold needs one configuration source, an effective date, and an auditable change history.

Verification status. The submission-time check is a release-verification item, not established by the client workflow description. Tests must exercise simultaneous submissions at the threshold.

11.4. Configurable Contributor Progress

The Dashboard uses `SegmentedProgress` to show category-level contribution. Depending on the collection task, progress may be expressed as contributed audio duration or as a percentage of an assigned voice or text target. The display is resumable across sessions and must derive from canonical contribution records rather than client-only counters.

11.5. Verbatim Transcription Policy

Transcribers must reproduce audio verbatim rather than rewrite it as standard prose. Guidelines are presented in-app and acceptance is recorded on the profile. Section 18.2 defines the canonical notation, dialect conventions, and handling of identifying content.

11.6. Configurable Correction Allowance

The permitted number of post-submission transcription corrections is deployment configuration; the Kikuyu pilot used three. The client displays the remaining allowance, while a database constraint, trigger, or controlled update RPC must reject changes beyond the active setting. Monitoring queries are useful but are not enforcement.

11.7. Single-Transcriber Editing Rights

Ordinary editing rights remain with the original transcriber; administrator moderation uses a separate privileged path. A second transcriber may not create a competing canonical transcription. The required enforcement combines:

- An owner-and-status RLS condition on `transcriptions`, represented by `auth.uid() = user_id AND status <> 'accepted'` for ordinary updates.
- A unique backend constraint on the canonical transcription's response identifier, preventing competing inserts under concurrency.
- Application-layer guards in the Transcribe page that provide early feedback and filter ineligible recordings without being treated as a security boundary.
- Administrator-only override paths for moderation.

12. Concurrency and Consistency Management

The `claim_random_transcription` RPC reserves transcription candidates in a transaction. Candidate selection and reservation occur together so concurrent requests cannot claim the same recording [18]. The documented sequence is:

1. Verifies the caller's `transcription_approved` flag.
2. Selects an eligible accepted recording without an active transcription or active lock.
3. Inserts a row in `transcription_locks` using the configured lease duration; the current deployment defaults to 10 minutes and the pilot used that value.
4. Returns the recording metadata (audio URL, prompt text, category).

Required identity control. The function must derive the claimant from `auth.uid()` or reject a supplied identity that differs from the authenticated caller. Earlier interface descriptions include an `_user_id uuid` parameter; its presence and validation must be checked against the tagged SQL definition. The exact deployed signature is not independently verified in this handbook.

A companion RPC `release_transcription_lock` clears the owner's lock on skip, logout, submission, or another documented abandonment path. An expired lock is reclaimable by a future caller. When a transcriber skips a candidate, the client temporarily excludes that recording from the same user's immediate subsequent random selections without making it globally ineligible. Keeping selection and lock creation in one database transaction addresses the principal double-assignment race, provided the function uses appropriate row locks, uniqueness constraints, caller validation, and concurrency tests.

The claim query should use a row-locking strategy such as `FOR UPDATE SKIP LOCKED`, backed by a uniqueness constraint on the locked response. The function must define whether a claimant with an existing active lock receives that same task or an error, whether one user may hold multiple locks, how candidates are randomized, how per-user skip exclusions expire, and how starvation is avoided. Submission must verify that the caller owns a still-valid lock and must insert or update the transcription and release the lock in the same database transaction.

Table 10: Transcription lock lifecycle.

State	Entry condition	Allowed exit
Available	Accepted recording has no canonical transcription and no active lock	Atomic claim creates an active lock
Claimed	Lock belongs to the authenticated approved transcriber and has not expired	Submit, skip, logout, administrative intervention, or expiry
Expired	Database time is at or beyond <code>expires_at</code>	Reclaim by another eligible transcriber or cleanup
Transcribed	Canonical transcription has been inserted	Administrative reopen only through an audited transition

Concurrency tests must run multiple database sessions against the same eligible pool and prove that no recording is returned to two claimants, no user can release another user’s lock, skip/logout release only the caller’s lock, expired candidates become reclaimable, temporary skip exclusion is user-scoped, and a retry after a network timeout is idempotent. Tests must execute against the configured lease rather than assume ten minutes globally.

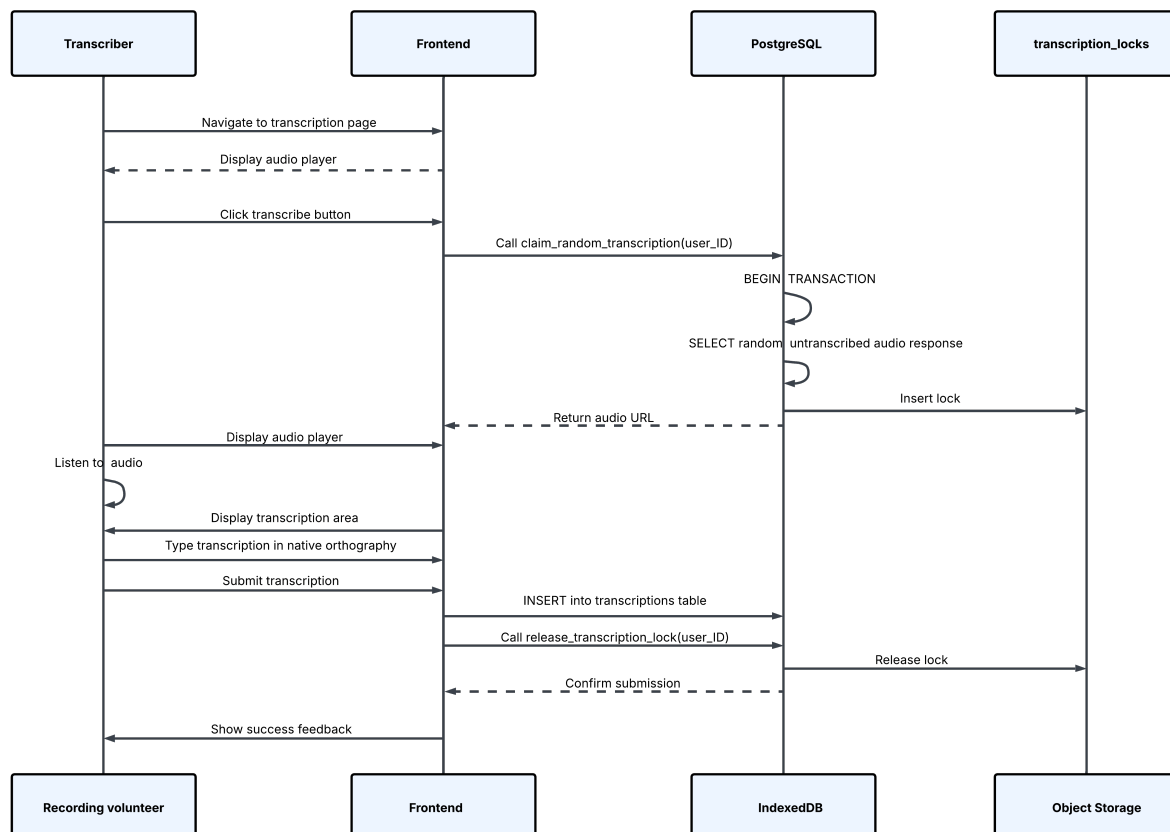


Figure 5: Sequence diagram for the atomic transcription claiming and submission process with pessimistic locking.

Database writes affecting a single critical transition—such as transcription submission plus lock release—should execute within explicit transactions. Object-storage upload and browser persistence

cannot participate in the same PostgreSQL transaction, so the recording workflow uses a documented saga: persist locally, upload using an idempotency key, insert or confirm the database row, update progress, and remove the local copy only after durable confirmation. Orphaned objects and incomplete rows require scheduled reconciliation rather than an unsupported claim of cross-service atomicity.

13. Voice Recording Pipeline

The recording pipeline integrates the browser **MediaRecorder** API [34], WebM/Opus capture, desktop waveform visualisation, mobile progress feedback, LocalStorage recovery, and asynchronous upload to object storage. The same capture path can present an open structured question for spontaneous response or a predetermined sentence for read speech.

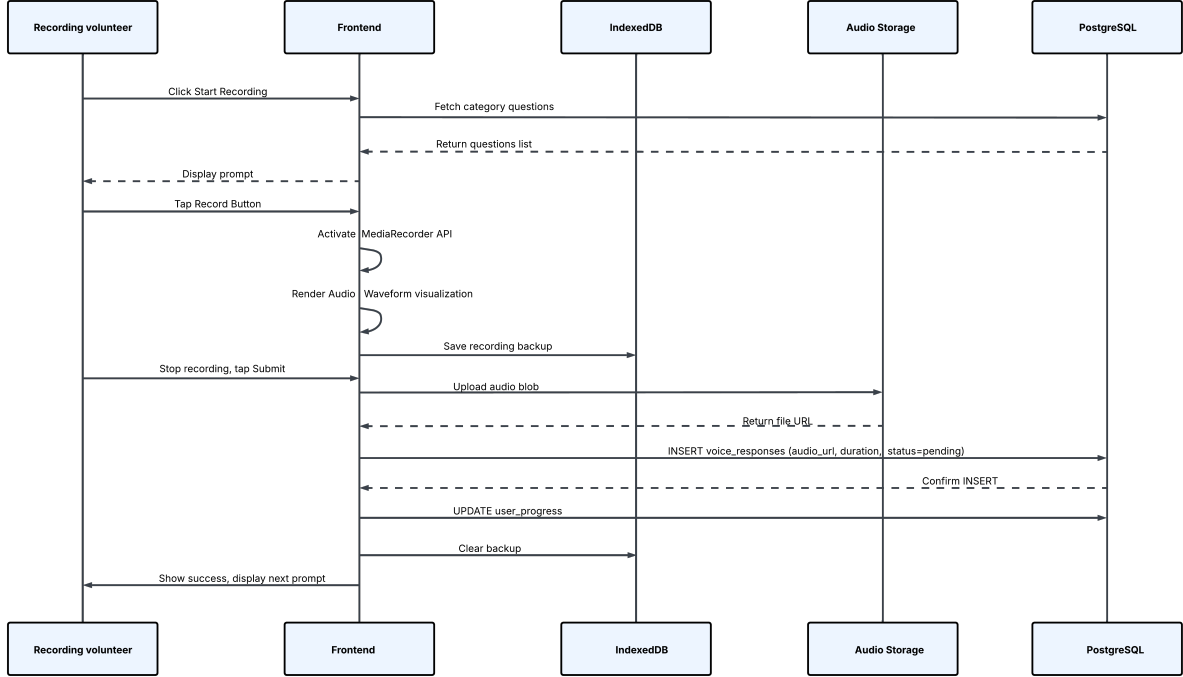


Figure 6: Sequence diagram of the voice recording pipeline from category selection through audio capture, local recovery backup, cloud upload, and database persistence.

The desktop **AudioWaveform** component renders a live waveform during capture. On smaller devices, **MinimalRecorder** replaces it with a circular progress indicator and high-contrast controls to reduce visual and GPU overhead. Category progress is presented through **SegmentedProgress**. A volunteer who is not conversant with the active prompt may skip it, and a completed recording can be played back before submission. On submission the WebM/Opus blob is uploaded to the **voice-recordings** bucket, a **voice_responses** row is inserted with status **pending**, **user_progress** is updated, the LocalStorage recovery copy is cleared only after confirmation, and the interface reports success.

Figure 7 shows the compact recording state used on a mobile viewport, including the active prompt, elapsed-time indicator, stop control, progress position, and explicit skip action.

13.1. Audio Contract

Release v1.0 records audio as WebM with the Opus codec through **MediaRecorder**. Opus is standardised for efficient interactive speech and audio over a wide range of bitrates [31]. The maximum duration is configurable; the pilot used 30 seconds. The implementation must test the actual MIME string selected by each supported browser and document sample rate, channel count, bitrate or quality setting, fallback behavior, maximum accepted byte size, active duration limit and tolerance, object-name convention, and metadata trusted from the client. Where a browser cannot produce the canonical format, the application must reject capture with actionable guidance or preserve the alternate format explicitly in the export manifest; silent format assumptions are not acceptable.

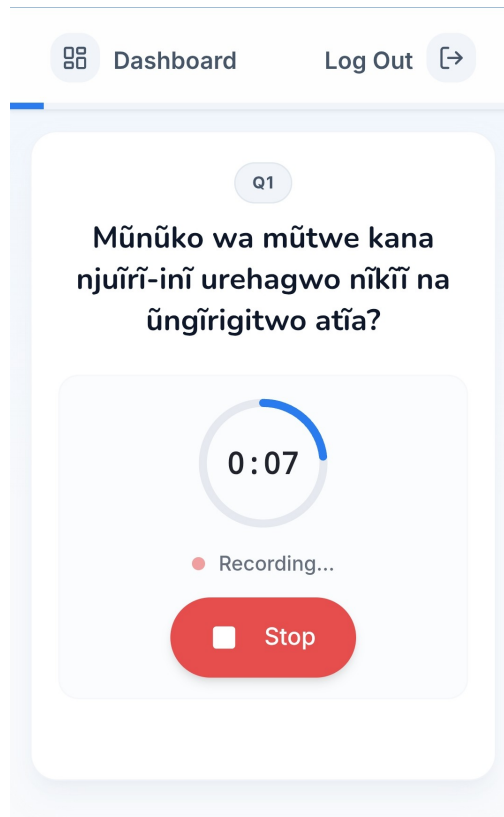


Figure 7: Mobile volunteer recording interface in release v1.0, showing a Kikuyu prompt, active recording timer, stop control, prompt progress, and skip action.

Each upload should carry a stable client-generated submission identifier so a retry can confirm an earlier success instead of creating a duplicate object. A checksum should be recorded or calculated where feasible to detect corruption. The database should store a storage bucket and object path, not depend permanently on an expiring signed URL. Signed download URLs should be generated only for authorized readers.

13.2. Recording Failure Matrix

Table 11: Required recording-pipeline recovery behavior.

Failure	Required behavior	Reconciliation
Permission denied	Preserve no unusable placeholder; show browser-specific recovery guidance	User grants permission and retries
Network loss before upload	Retain the bounded Base64 recording and metadata through <code>recordingStorage</code>	Let <code>RecoveryDialog</code> resume or retry on the next visit
Upload succeeds, row insert fails	Retain submission ID and do not announce completion	Retry insert or remove orphan after retention window
Row exists, client misses response	Query by idempotency/submission ID before retrying	Treat existing matching row as success
Progress update fails	Keep recording durable and report partial completion	Recompute progress from canonical responses
Local quota exceeded	Stop recording safely and explain storage remediation	User frees storage or submits online

14. Backend Interface Reference

This section summarises documented RPC purposes, intended callers, and required failure handling. It is not a copy-ready API specification: exact parameter names, return schemas, and deployed grants have not been verified against an application checkout. Use the tagged SQL definitions and generated TypeScript types for call signatures. Section 22.2 defines the evidence needed to complete the release reference.

14.1. Core RPC Contracts

Table 12: Documented RPC behaviour and required failure handling; exact signatures require release verification.

RPC	Intended caller	Documented purpose	Required failure behavior
<code>claim_random_transcription</code>	Authenticated transcriber	Reserves an eligible accepted response and returns recording metadata with a configured lease	Not authenticated, not approved, no work, or conflicting state; no partial lock
<code>release_transcription_lock</code>	Authenticated lock owner	Releases the caller's claim on a documented skip, logout, submission, or abandonment path	Idempotent if already released; cannot release another user's lock
<code>get_question_counts</code>	Verified user or administrator	Counts used for progress and prompt availability	Stable empty result; must not expose restricted prompts
<code>get_question_unique_user_counts</code>	Verified user or administrator	Unique respondent count per permitted question	Same filtering and threshold semantics as submission path
<code>get_admin_chart_data</code>	Administrator only	Aggregate operational data with a documented time zone and interval	Authorization failure without leaking aggregates

Error-handling requirement. Do not expose raw PostgreSQL error text as a stable client API. Map expected failures to the vocabulary in the companion runbook's Error Catalogue, including `TRANSCRIBER_DENIED`, `NO_AVAILABLE_WORK`, `LOCK_EXPIRED`, and `TRANSCRIPTION_CONFLICT`. These are documentation-level codes, not verified wire responses. Unexpected failures should carry an internal correlation identifier.

14.2. Edge Function Contracts

The documented `reset-password` function supports password recovery. The `migrate-drive-audio` and `upload-to-gcs` functions belong to legacy storage operations rather than ordinary contributor workflows. Their exact HTTP contracts and current deployment status are not established here.

Access requirement. Migration functions must be restricted to authorised administrative use. Legacy functions should be disabled or removed once their retention requirement ends. Before invoking any of these functions, verify its current authentication, request schema, and operational limits using the release reference.

15. Storage Strategy and Migration

The documented release stores canonical audio in the Supabase `voice-recordings` bucket. Question records may contain an `image_url`; the deployment must document the hosting and authorization model for those prompt images rather than assuming a second public bucket. Three Deno Edge Functions support specialised flows:

- `reset-password` — handles password-reset email dispatch with custom branding.
- `migrate-drive-audio` — ferries legacy recordings stored in Google Drive into the canonical `voice-recordings` bucket.
- `upload-to-gcs` — a server-side proxy used during early experimentation to upload directly to a Google Cloud Storage bucket; retained for archival workflows.

Two administrative tools complement the edge functions:

- The **Audio Storage Migration** tool moves legacy Drive-hosted audio into Supabase storage in batches.
- The **Drive Backfill** tool reconciles `voice_responses` rows whose `audio_file_url` is missing or broken by matching them to Drive files via a deterministic naming convention.

15.1. Storage Authorization and Object Lifecycle

Bucket documentation must specify whether a bucket is public or private, permitted path prefixes, upload owner, read audience, signed-URL lifetime, MIME and size restrictions, and deletion behavior. The canonical audio bucket should normally be private. A transcriber must receive only temporary access to the recording actively claimed by that transcriber; bucket-wide listing is not an acceptable authorization shortcut.

Object paths should be opaque and stable, for example a generated submission UUID rather than an email address or human name. Database deletion and consent withdrawal must define how the corresponding object is located and removed. A scheduled reconciliation job should report database rows without objects, objects without rows, duplicate submission identifiers, inaccessible objects, and checksums that no longer match.

15.2. Cloud Providers and Transfer Review

Submitted recordings and linked records reside in managed cloud services, not exclusively on contributors' devices. The supplied policy names Supabase and Vercel but does not specify deployment regions or the safeguards for each international transfer. Before release, maintainers must record actual storage, hosting, backup, and recipient locations and obtain approval for the applicable transfer arrangements and contractual restrictions. Do not infer a Kenyan hosting location from the controller's address. Legacy Drive or Google Cloud migration paths need the same inventory and review if still used.

15.3. Retention and Deletion

The supplied policy's retention table is explicitly a **draft sample schedule**, not a statement of approved JasinaHub practice or statutory deadlines. Its illustrative periods must not be copied into production deletion jobs or published as commitments without approval. Retention remains limited by the stated purposes, consent, approved protocol, and any earlier erasure obligation.

An approved schedule must cover account/contact records; consent, withdrawal, and rights requests; rejected or abandoned recordings; accepted audio, transcripts, and metadata; activity/presence and audit/security logs; local recovery copies; backups; recipient exports; and research outputs or models. For each category, record the retention trigger, period, responsible owner, deletion or de-identification method, documented exceptions, and evidence that the process works. Review continued necessity and verify that approved deletions cover both database rows and audio objects.

15.4. Migration Procedure

Before migrating legacy audio, record the source inventory, destination naming rule, expected count and total bytes, credential scope, and a rollback or re-run strategy. Migration must be idempotent: a repeated batch should compare a stable source identifier or checksum rather than upload a second copy. After each batch, verify count, size, checksum where available, database references, and authorized playback. Preserve a machine-readable manifest and do not delete the source until the retention and verification period has elapsed.

16. Offline Resilience and Recovery

When a connection drops, contributors need to know whether their work is saved and how to continue. JasinaHub uses PWA caching and a local recording copy to support recovery [10].

- **Service Worker / PWA shell:** The application installs as a PWA with an offline-capable shell, supporting installation to the home screen on Android, iOS, and desktop.
- **Local recording copy:** Before upload, `recordingStorage` serialises one bounded WebM/Opus blob as Base64 in LocalStorage.

- **Recovery Dialog:** On the next visit, `RecoveryDialog` detects the retained recording and lets the user resume or retry the upload.
- **Offline indicator:** `OfflineIndicator` exposes current network state with accessible colour and ARIA semantics.
- **Interaction support:** `useHapticFeedback`, `useSwipeGesture`, and `usePullToRefresh` provide mobile-oriented feedback and navigation without changing the authoritative backend state.

Recovery limits. Cached interface resources and a retained recording do not provide fully offline authentication, collection, transcription, or submission. `LocalStorage` is synchronous, uses additional space for Base64, and is subject to quota limits and browser eviction. Release v1.0 supports one recoverable recording, not a general multi-item queue.

16.1. Browser Storage and Privacy

The privacy notice must distinguish GoTrue session storage, `LocalStorage` audio recovery, unfinished transcript storage, service-worker interface caches, and server-held contributions. Recovery copies may remain until handled, removed, or cleared or evicted by the browser. Clearing site data can sign a participant out and destroy unsent work, but does not delete submitted recordings, transcripts, or linked records from the server. Server-side erasure requires the rights-request process in the operations runbook.

The supplied policy does not promise a telemetry-disable setting. It describes no advertising cookies or third-party behavioural advertising in the deployment; introducing non-essential tracking requires policy review and consent where required. These statements must be checked against the actual browser integrations before publication.

16.2. Recovery Record Contract

The recovery record should identify the submission, its owner and prompt, the Base64 payload, MIME type, duration, creation time, retry count, and last error. The client must distinguish `local`, `uploading`, `confirmed`, and `failed` states. It must retain the payload until the server confirms that both the audio object and its database record have been saved.

Retries should use bounded exponential backoff with jitter and an idempotency key. Logging out must not silently submit one user's retained audio under another session. The user must be able to inspect and delete an abandoned local recording. Document retention and secure deletion expectations, private-browsing limitations, `LocalStorage` quota behavior, and multiple-tab coordination. A future migration to IndexedDB is recommended before supporting multiple queued recordings or larger offline workloads.

17. Security and Access Control

Security in JasinaHub is enforced primarily at the database level using PostgreSQL Row-Level Security. Authorisation policies are therefore consistently applied regardless of client behaviour.

- **Role storage:** Roles live in `user_roles`, never on `profiles`, eliminating profile-update privilege escalation paths.
- **`has_role()` as SECURITY DEFINER:** Encapsulates role checks inside a function whose definer privileges bypass policy recursion, enabling clean RLS expressions across all admin policies.
- **Authentication:** Email-and-password authentication only, deliberately excluding social providers to maximise accessibility for contributors who do not maintain external identity-provider accounts. JSON Web Tokens are validated by the database engine, minimising the trusted computing base.
- **Password recovery:** The `supabase/functions/reset-password` Edge Function supports the dedicated recovery flow and must enforce approved redirect origins and non-enumerating responses.
- **Administrator verification:** A `verified` flag set by an administrator restricts access to prompts and collection workflows after account registration.
- **Transcriber gate:** A separate `transcription_approved` flag governs access to the transcription pipeline.

- **Feedback data minimisation:** The `feedback_submissions` table contains no direct `user_id` column. Operational metadata may still permit indirect correlation, so privacy notices and retention controls must not promise absolute anonymity.

17.1. Threat Model and Security Boundaries

The security model must assume a modified browser client, stolen ordinary-user credentials, malicious file uploads, replayed requests, concurrent requests, leaked signed URLs, and accidental administrator actions. Principal trust boundaries are the browser, Supabase API gateway, PostgreSQL/RLS layer, private object storage, Edge Functions, third-party migration services, and administrator workstations.

At minimum, security review must cover:

- RLS and storage-policy bypass attempts for every role and state.
- Cross-user ownership changes and insecure direct object references.
- `SECURITY DEFINER` function ownership, `search_path`, grants, and caller identity.
- Upload MIME validation, file-size limits, object-name sanitisation, and content handling.
- Authentication rate limiting, password policy, session lifetime, token refresh, and account recovery.
- Secret storage, rotation, least privilege, and prevention of server secrets entering frontend bundles or logs.
- Administrator action auditability and protection of destructive exports, migrations, and deletions.
- Dependency, container, and database migration review in continuous integration.

17.2. Privacy Policy, Controller, and Consent

The supplied privacy policy identifies Dedan Kimathi University of Technology, acting through DSAIL, as the controller for the hosted website and PWA. Privacy, participation, security, and rights requests go to dsail-info@dkut.ac.ke, with referral to the University's designated data protection office as needed. These are the hosted Service's responsibilities; another deployment must identify its own controller and approved notice.

Before recording, volunteers receive the policy at <https://jasinahub.vercel.app/privacy-policy> and the applicable electronic consent form. Capture cannot begin until `voice_recording_consent` and `consent_timestamp` are recorded. Preserve the accepted policy and consent versions alongside that evidence. Approved transcribers additionally accept the guidelines through `transcription_guidelines_agreed` and `transcription_guidelines_agreed_at`.

Consent must explain recording activities, expected participation, managed-cloud storage, compatible future research, controlled sharing, pseudonymisation, and residual voice-identification risk. It must also explain the choices to decline, stop, withdraw, access contributions, close an account, and request erasure. Participants must not record others without authority or include private identifying details. Suspected under-age participation must be escalated for review and removal of children's data collected without appropriate authorisation, as described by the policy.

17.3. Information and Permitted Purposes

The policy covers account and contact details, demographic and dialect attributes, verification and approval states, contributions and corrections, consent records, feedback, and linked operational records. Registration must distinguish required from optional fields. Passwords belong to the authentication flow and must not be copied into application logs or exports. Clinical records are outside this collection workflow.

The supplied policy distinguishes the following purposes and stated grounds; this is a policy summary, not an independent legal determination:

- Requested account administration and support: steps necessary to provide the requested Service.
- Contribution collection, research, and compatible model development: explicit informed consent and the applicable sensitive-data condition.

- Security, task coordination, quality assurance, and improvement: operational, research, and security interests assessed against participant rights, without overriding research consent.
- Required records and regulatory responses: applicable legal obligations, alongside institutional and research-governance requirements.
- Controlled research sharing: participant consent and documented data-access conditions.

Future use is limited to the compatible speech, language, data-quality, accessibility, and related research explained when consent was obtained. The policy requires a new lawful basis and fresh consent where required for materially incompatible use, public release of identifiable recordings, or broader sharing. It also calls for DSAIL to complete or review a Data Protection Impact Assessment before new high-risk processing or material changes and to record the applicable sensitive-data condition. Permission for downstream research does not add model training to the JasinaHub software boundary.

17.4. Sharing, Human Review, and Participant Rights

The policy names Supabase for authentication, database services, functions, and audio storage, and Vercel for web hosting and delivery. Other recipients include authorised staff and transcribers, approved research collaborators, institutional oversight personnel, and authorities where disclosure is lawfully required. Access must remain limited to the authorised purpose. Funding or an open-source software licence does not grant access to participant-level data. The policy excludes personal-data sales and unrelated commercial advertising using participant contact details.

Progress calculations, prompt retirement, candidate selection, lock expiry, and recovery detection are automated workflow functions. Administrators verify participants, approve transcribers, review contributions, and authorise exports. The policy describes no legal or similarly significant participant decisions made solely by automated means and provides a route to request reconsideration.

Rights requests can concern recordings, transcripts, account information, or linked activity records, not just contact details. They include access, correction, objection, restriction, consent withdrawal, eligible erasure, portability where applicable, and information or human review concerning significant solely automated decisions. The companion `operations-runbook.pdf` describes intake, identity checks, response handling, and withdrawal across active systems, exports, backups, and downstream research.

Removing names does not make voice anonymous: acoustic signals, demographics, and response content may still identify a speaker [30]. The policy distinguishes voice identifiability from processing voice for unique identification, which may constitute biometric processing. The collected language, consent, prompt, and provenance metadata can support a dataset data statement [9]; neither documentation nor these safeguards alone establishes compliance with Kenya’s Data Protection Act [26].

18. Quality Assurance Subsystems

Quality control occurs at two stages. Administrators first review recordings before transcription to exclude unusable audio. After transcription, a second review compares each transcript with its source recording and verifies that the written text follows the project conventions and accurately represents what was spoken.

18.1. Response Verification

Administrators triage `voice_responses` rows in a dedicated panel that supports debounced server-side search, optimistic UI updates, accept/reject actions, an inline audio player, and per-user accepted/rejected duration roll-ups (rendered as emerald/red summary cards).

A recording is accepted when it is intelligible, relevant to the assigned prompt, primarily in the target language, free from directly identifying content, and sufficiently distinct from existing submissions. Review is mode-aware: a structured spontaneous response is checked for relevance, while read speech is also checked against the predetermined sentence without treating natural accent or dialectal realisation as an error. Minor background noise, natural pauses, fillers, laughter, repetitions, or code-switching do not by themselves require rejection when the response remains usable. Table 13 defines the operational rejection vocabulary used during the pilot.

Table 13: Recording rejection criteria used during administrative quality review.

Reason	Operational interpretation
Excessive noise	Noise, distortion, clipping, or competing speech prevents reliable listening or transcription.
Silence	The clip contains no meaningful spoken response or is dominated by silence.
Wrong language	The response is not primarily in the target language; limited natural code-switching remains acceptable.
Off-topic response	The response does not address the assigned prompt or contains unrelated speech.
Personally identifying content	The recording discloses information that may directly identify the speaker or another person.

The broader administrative command centre is shown in Figure 8. It consolidates recording, duration, transcription, user-verification, and review-backlog indicators with trend visualisation and navigation to operational modules. The displayed values are a point-in-time application snapshot and are not the authoritative pilot totals in Table 14.

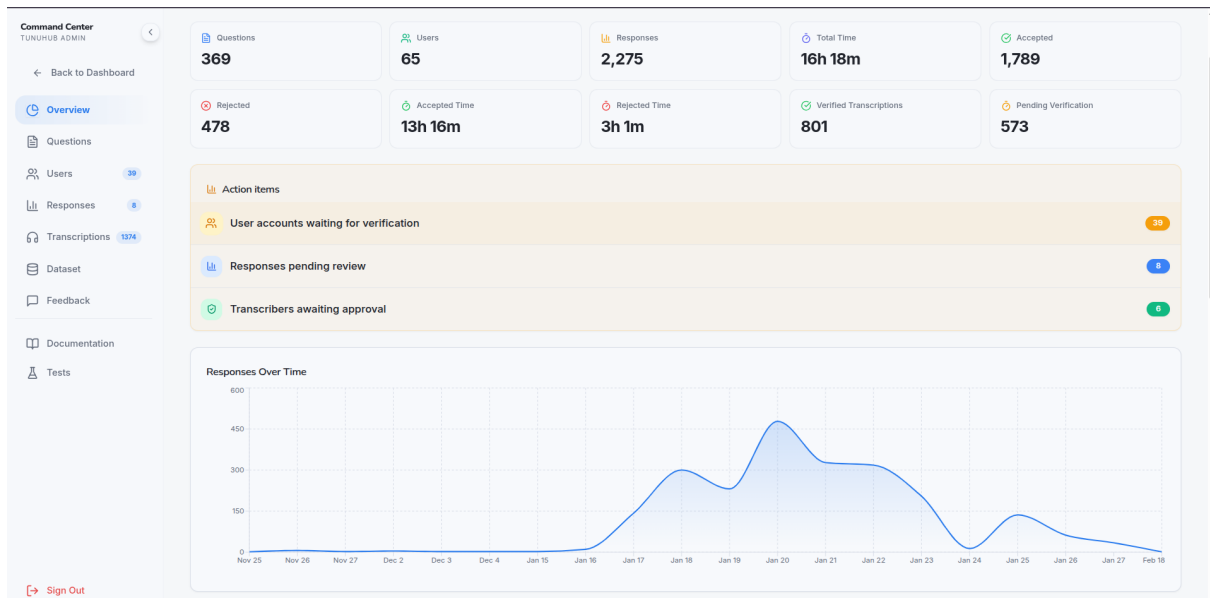


Figure 8: Administrator command-centre overview showing collection statistics, action queues, navigation to management modules, and a response trend chart. Snapshot counts reflect the state of the running platform when captured.

18.2. Transcription and Transcript Validation

The transcription review surface presents the audio, prompt, submitted text, edit count, and status together. Administrators compare the submitted text with the original recording and may accept, reject, or amend transcriptions; the `validated_by` and `validated_at` columns capture the audit trail. A speech-transcription pair becomes export eligible only after both the recording and transcript have passed their respective review stages. Trained transcribers and reviewers use the same standardised guide, available after transcription access is granted, so that fidelity is assessed against a common convention rather than individual preference. The guide requires:

- retaining fillers as spoken using bracketed forms such as `[mmh]`, `[eeh]`, and `[aah]`, and marking noticeable pauses as `[pause]`;
- using `[?]` for segments that cannot be heard or understood reliably rather than guessing;
- retaining code-switched words as spoken and enclosing each non-Kikuyu word with `[cs]` markers, for example `[cs]hospital[cs]`;

- applying ordinary sentence punctuation and capitalisation without adding emphasis, and writing numbers in words as spoken;
- marking cut-off words with a trailing hyphen and preserving repetitions;
- retaining the speaker’s dialectal form under the agreed orthography instead of normalising it to another variety; and
- replacing direct identifiers in text with [personal information] and flagging the associated recording for administrative review.

Recent Kikuyu ASR error analysis found noisy reference transcriptions to be a major source of high-error cases, reinforcing the need for these shared conventions and auditable review [3].

The transcriber workspace enforces an attentive-listening sequence. Audio may be replayed and its playback speed adjusted, but transcript-entry and submission controls remain disabled until the assigned recording has played to completion at least once. Draft text is retained and restored after a connectivity interruption. Skipping releases the lock and temporarily suppresses the candidate for that transcriber; logout and lease expiry also return unfinished work to the queue. The correction allowance is configurable and was set to three during the pilot.

Figure 9 illustrates the transcriber-facing workspace. Instructions and the original question remain visible while the user controls playback and enters the verbatim text, reducing the need to navigate away from an active lock.

Figure 9: Transcription workspace showing inline instructions, guideline access, the original Kikuyu prompt, audio playback controls, the verbatim text field, and submission action.

18.3. Dataset Curation and Export

Administrators use the Dataset tab to export accepted audio–text pairs as a ZIP archive with a CSV manifest. The exporter downloads audio in batches and retries temporary failures with increasing delays. If it cannot retrieve the audio, it can produce a metadata-only CSV; the interface must identify this output as incomplete.

The export replaces contributor names with stable project-generated identifiers such as TH-4F2A9C1B, following the TH-XXXXXXX format. Recordings from one contributor remain linked by the same identifier. The export must not expose the identity mapping or any secret used to generate it.

Recipient checks. Before release, an administrator must confirm that the recipient’s purpose, permitted fields, access period, and sharing conditions fit the participant consent and data-access agreement. Record who received each dataset version so later restrictions or erasure requests can be communicated. Software access under the MIT licence, research funding, or removal of names does not authorise public release or unrestricted data reuse. More restrictive project-specific consent takes precedence.

Marking transcript text as [personal information] does not remove an identifier spoken in the audio. Flag the source recording for review and exclude it from export until the identifying content has been resolved through approved redaction where practicable or rejection.

Each export needs a version and a documented manifest format so researchers can identify its contents and interpret each field. The manifest should include recording ID, relative audio path, language and dialect, prompt and category identifiers, prompt mode, transcript, duration, codec/MIME type, and review status. It should also include pseudonymous contributor and transcriber identifiers where permitted, creation and validation times in UTC, consent-policy version, and licence or usage restrictions. Direct contact details must not be exported.

The ZIP should include a machine-readable manifest, dataset-level metadata, schema version, generation timestamp, source software release, record count, total duration, and checksums. Export queries must use a transactionally consistent snapshot or record the cut-off time. Cancelled consent, rejected media, unaccepted transcriptions, missing objects, and failed checksum validation must exclude or fail the affected records according to an explicit policy rather than silently producing a partial dataset.

Listing 7: Illustrative export manifest record.

```

1 {
2   "schema_version": "1.0",
3   "recording_id": "<uuid>",
4   "audio_path": "audio/<uuid>.<extension>",
5   "language": "<BCP-47-or-project-code>",
6   "dialect": "<controlled-value>",
7   "prompt_mode": "<spontaneous-or-read>",
8   "transcription": "<verbatim-text>",
9   "duration_seconds": 12.4,
10  "mime_type": "<recorded-mime-type>",
11  "contributor_id": "TH-4F2A9C1B",
12  "transcriber_id": "<pseudonymous-id>",
13  "validated_at": "<UTC-ISO-8601-timestamp>"
14 }
```

18.4. Anonymous In-Product Feedback

The policy uses “anonymous” for feedback stored without a direct account identifier. This is a data-minimisation measure, not a guarantee against correlation through timestamps or infrastructure logs. Ask participants to avoid identifying details in free-text answers, and apply the approved access and retention rules to both raw feedback and feedback exports. A feedback subsystem enables continuous platform improvement:

- Admins author questions of type `rating` (1–5 stars), `nps` (0–10), `choice_single`, `choice_multi`, `text` (open-ended, max 2000 characters, validated with Zod), and `boolean`.
- Any authenticated user may submit responses via a dedicated `/feedback` page accessible from the Dashboard navigation.
- Release v1.0 persists each answer in `feedback_submissions` with its question, answer value, and submission time, but without a direct `user_id`.
- `AdminFeedbackTab` presents administrator feedback review and aggregate analysis, including rating/NPS and choice distributions, boolean summaries, searchable open-text answers, and per-question CSV export where implemented.

Figure 10 shows the management view for feedback questions, including question type, mandatory status, activation state, editing, deletion, and question creation. This is an administrative feedback configuration screen rather than the health-prompt catalogue used for speech collection.

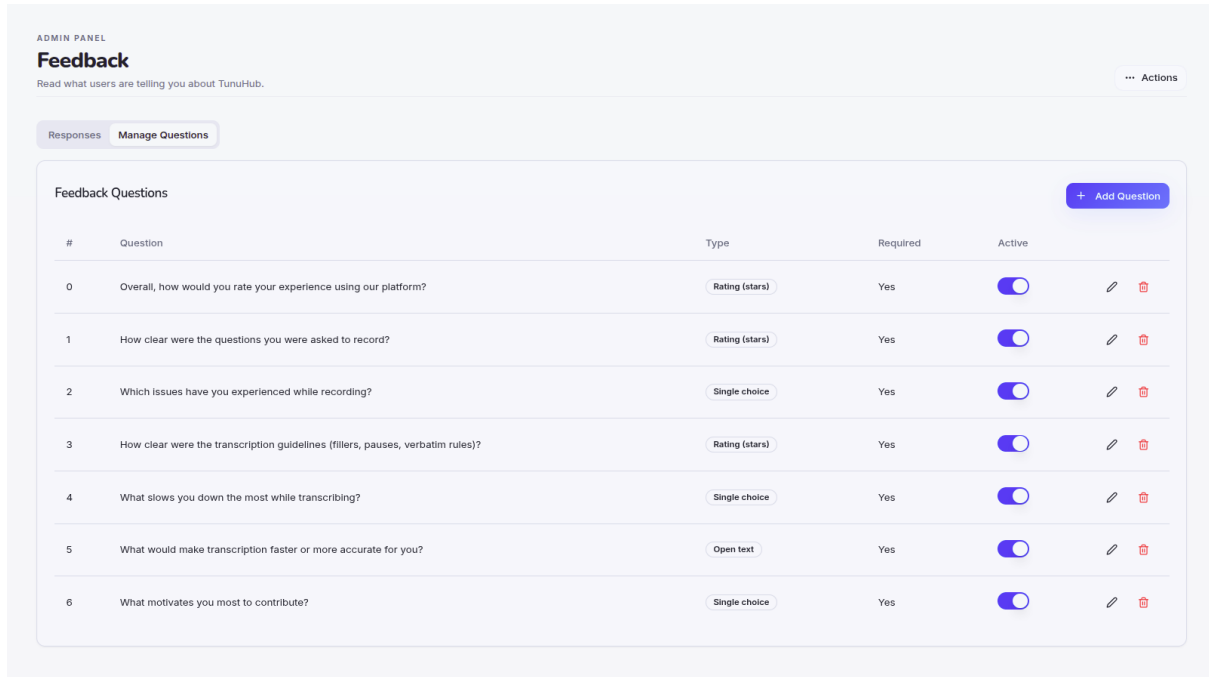


Figure 10: Administrator feedback-question management interface showing ordered questions, response types, required flags, activation controls, and editing actions.

19. Pilot Deployment and Dataset Statistics

The pilot assessed whether JasinaHub could support the complete corpus-creation workflow—from prompted recording and upload to quality review, transcription, curation, and export—under practical deployment conditions. It was not designed as a formal usability study or an evaluation of ASR performance. Release v1.0 was piloted with 25 adult, fluent Kikuyu speakers and an inventory of 369 health-related prompts. Kikuyu was selected because the 2019 Kenya Population and Housing Census identifies the Kikuyu as Kenya’s largest ethnic community [21]. AfriVoices-KE also reports Kikuyu as the largest of its five language subsets, with 754 hours from 1,331 contributors [33]. The research team’s base in Nyeri enabled direct engagement with speakers, while the software remained configurable for other languages and domains.

Participation was voluntary and participants received compensation for their time. Eligible participants were at least 18 years old. The group included speakers from four regional varieties: *Kĩ-mũrang’a* from Murang’a, *Gi-gabete* from Kiambu, *kĩ-ndia/gĩ-gĩchũgũ* from Kirinyaga, and *kĩ-mathĩra* from Nyeri. Before recording, each participant reviewed an electronic informed-consent form covering project purpose, expected participation time, recording activities, secure storage, potential future research use and controlled sharing, pseudonymisation, and residual voice-identification risk. The form also described the rights to decline or stop participation, withdraw consent, access contributions, delete an account, and request erasure.

Recording ran from 16 January to 18 February 2026, followed by transcription from 17 March to 14 May 2026. Counts refer to individual audio submissions distributed across the prompts: each prompt could receive several speakers’ responses, and each participant could answer several prompts. Five participants also served as trained, approved transcribers, making the transcription group a subset of the recording contributors.

The three-response retirement rule was introduced after pilot collection had already begun. Earlier submissions were retained, so some prompts accumulated more than three responses and the final total was not limited to 369×3 recordings. This historical exception must not be mistaken for a defect in the current retirement rule.

Table 14: JasinaHub pilot deployment and quality-review outcomes.

Pilot metric	Value
Participants	25
Approved transcribers	5
Distinct health-related prompts	369
Recordings submitted	2,275
Total collected duration	16 h 18 min
Accepted recordings	1,789 (78.6%)
Accepted recording duration	13 h 16 min (81.4%)
Completed transcriptions	1,789
Completed speech–transcription pairs	1,789
Excluded recordings	486
Excluded audio duration	3 h 02 min

Administrators first reviewed the recordings and then performed a second quality-assurance review of completed transcripts against their source audio. The workflow retained 1,789 recordings and excluded 486 under the criteria in Section 18.2. All retained recordings were transcribed, yielding 1,789 completed speech–transcription pairs. The difference between the 78.6% recording-count acceptance rate and 81.4% duration retention indicates that the retained clips were slightly longer on average. These figures demonstrate operational use of collection, review, transcription, curation, and export; they do not by themselves establish usability, ASR quality, population representativeness, transcription agreement, or production-scale capacity. The curated pairs are being used by the separate *Jasinah* project to develop a Kikuyu ASR model.

For the pilot, curated export access remained subject to project data-governance and authorization procedures, the published privacy policy at <https://jasinahub.vercel.app/privacy-policy>, and safeguards applicable to personal and biometric data under Kenya’s Data Protection Act [26]. Contributor names were replaced with stable TH-XXXXXXX pseudonyms so same-speaker records could remain linked without a direct name. The dataset was not treated as anonymous because speech signals may retain identifying characteristics [30].

20. Impact and Reuse Potential

The pilot produced a reviewed corpus through community participation in recording and transcription. This approach is consistent with participatory African-language research principles [23].

Reuse in another community requires a review of language metadata, transcription conventions, consent, cultural appropriateness, data residency, moderator capacity, and the intended downstream licence. These considerations determine whether the configuration model described in Section 3 is sufficient or needs extension.

The reviewed pairs are being used to develop *Jasinah*, providing a concrete use for the pilot output.

20.1. Limitations

The pilot tested the corpus-creation workflow, not ASR performance or wider effects on digital inclusion. Evaluating the *Jasinah* model requires a separate study.

The pilot involved only 25 Kikuyu-speaking participants and health-related prompts, so it does not establish usability, speech diversity, or data quality across other languages, communities, or domains. Administrators and transcribers used a shared quality guide, but inter-annotator agreement was not measured. Pseudonymisation reduces exposure of direct identifiers but cannot eliminate re-identification risks arising from voice characteristics, demographic metadata, or response content. Dataset access must therefore remain controlled under the project’s consent, privacy, and authorisation procedures.

21. Observability and Testing

JasinaHub treats observability as a first-class platform capability.

21.1. Activity Logs

The `useActivityTracker` hook appends significant actions—including page visits, recording submissions, and transcription completion—to `user_activity_logs`, capturing action name, page context, and a JSON `details` payload. Administrators consume the stream through a Supabase Realtime channel for near-live monitoring.

Audit records should include actor, action, target type and identifier, outcome, UTC timestamp, environment, and a correlation identifier while excluding secrets and unnecessary personal content. Append-only behavior must be enforced by policy, retention must be defined, and administrator reads should themselves be auditable where required.

Under the supplied privacy policy, activity, presence, browser/device information, network state, and error/security records may be linked to accounts; they must not be treated as necessarily non-personal telemetry. Include them in the data inventory, rights-request searches, and approved retention schedule. Avoid logging raw transcript text, contact details, access tokens, or signed audio URLs, and report aggregate participation statistics without unnecessary individual detail.

21.2. Realtime Presence

The `usePresenceTracking` hook and `PresenceTracker` component publish per-user online state to a Supabase Realtime channel, enabling administrators to see active contributors. The dashboard uses Recharts for recording trends, transcription progress, response distributions, and engagement summaries.

21.3. In-App Test Runner

The Admin panel includes a browser-side "Tests" tab organised into thirteen TypeScript-defined suites: **Smoke**, **Auth**, **RLS**, **RPCs**, **Data Integrity**, **Business Rules**, **Storage**, **Performance**, **Accessibility**, **Compatibility**, **End-to-End**, **Resilience**, and **Regression**. Together they cover:

- **Authentication and Roles:** session validity, `has_role` verification, `user_roles` self-only visibility.
- **RLS and Data Access:** admin read paths, anonymous-client lockout for `voice_responses`, `transcriptions`, `questions`, `user_roles`.
- **RPCs:** `get_question_counts`, `get_question_unique_user_counts`, `get_admin_chart_data`, plus a destructive `claim/release` round-trip.
- **Storage:** bucket listability and reachability of a sampled accepted recording URL.
- **Business Rules:** active duration limit, correction allowance, prompt-retirement threshold, playback gate, and lock-lease assertions executed against controlled data.
- **Edge Functions, Performance, Compatibility, Accessibility, Data Integrity:** additional suites covering function reachability, latency budgets, browser-feature presence, ARIA conformance, and referential-integrity spot checks.

The runner provides useful authenticated smoke checks, but it is not a substitute for isolated automated tests. Destructive cases must use dedicated fixtures and must not claim or modify ordinary production work. Production smoke tests should be read-only wherever possible and clearly label every mutation before execution.

Figure 11 shows the runner's aggregate status and expandable suite results. Pass rate, failures, skipped tests, coverage, and timings are properties of the captured execution and must not be interpreted as a permanent release guarantee.

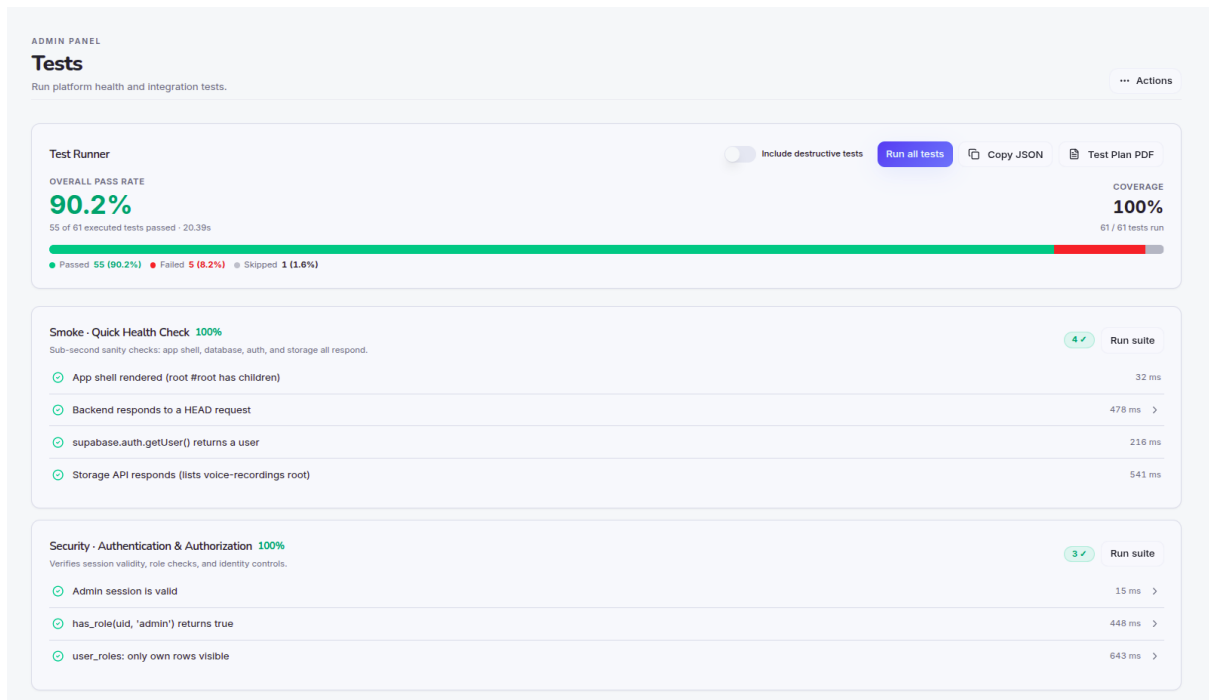


Figure 11: Administrator test-runner interface showing point-in-time pass rate, executed-test coverage, destructive-test control, and individual Smoke and Authentication/Authorization checks.

21.4. Automated Test Strategy

Table 15: Required automated test layers.

Layer	Coverage	Execution point
Unit/component	Validation, state reducers, recorder controls, formatting, retry logic	Every pull request
Database/RLS	Constraints, triggers, policies, grants, privileged functions	Every migration and pull request
Integration	Supabase client, storage, RPC, authentication, offline queue	Pull request against isolated local backend
Concurrency	Parallel claims, skip/logout release, user-scoped exclusion, expiry, idempotent retry	Pull request and release candidate
End-to-end	Signup, administrator verification, spontaneous/read recording, recovery, moderation, gated transcription, export	Release candidate and staging
Accessibility	Keyboard navigation, names/roles, focus order, contrast, automated WCAG checks	Pull request and manual release review
Performance/load	Upload latency, claim contention, export size, dashboard queries	Before capacity-changing releases
Recovery	Backup restoration, failed migration, orphan reconciliation, offline recovery	Scheduled operational exercise

The repository must publish commands for each layer and state whether tests mutate data, which fixtures they require, their expected duration, and their pass threshold. Continuous integration should install from the frozen lockfile, lint, type-check, build, start an isolated backend, reset migrations, run tests, and verify that generated database types are current.

21.5. Operational Metrics and Alerts

Product activity is not sufficient production monitoring. Operators should track authentication failures, client exceptions, upload success and latency, queued-upload age, orphaned-object count, RPC error rate,

lock contention and expiry, moderation backlog, Edge Function failures, database resource use, storage growth, and export failure rate. Every alert needs an owner, threshold, severity, dashboard link, and runbook. Service objectives should be defined after collecting a baseline rather than claiming unmeasured scalability or availability.

22. Documentation Maintenance and Release Checklist

Documentation is part of the release, not a retrospective artifact. The pull-request template should require documentation updates when a change affects setup, configuration, dependency versions, schema, RPCs, RLS, storage, workflow states, export format, deployment, or operational behavior.

22.1. Documentation Source Layout

The documentation is maintained as code and produces two audience-specific outputs from shared sources. `main.tex` builds the architecture and developer guide; `operations-runbook.tex` builds the deployment, backup, incident-recovery, privacy-request, troubleshooting, glossary, and error-reference companion. Chapters live in `sections/`; cross-cutting reference material lives in `appendices/`; bibliographic metadata lives in `references.bib`; application screenshots live in `images/`; and editable diagram descriptions live in `diagrams/`. Contributors should modify the smallest relevant source file and must compile every output that includes it.

Build the main guide with `latexmk-pdf-interaction=nonstopmode-halt-on-errormain.tex` and the companion with `latexmk-pdf-interaction=nonstopmode-halt-on-erroroperations-runbook.tex`. Afterward, inspect both logs for undefined citations, undefined references, multiply defined labels, overfull boxes, and other material warnings. If a future release adds a Makefile or continuous-integration workflow, this section must record the checked-in path and exact commands only after those artifacts exist.

22.2. Interface Documentation Evidence

For each public RPC, record the tagged SQL definition and generated type, caller permissions, exact parameters, return schema, side effects, idempotency, failure mapping, transaction boundary, and a tested example. Confirm identity validation and skip-exclusion behaviour rather than inferring them from a function name or client component.

For each Edge Function, record deployment status, HTTP method, authentication, request and response schemas, CORS behaviour, size and time limits, retries, rate limits, required secret names, and audit events. Do not include secret values. Associate verified entries with a source commit, configuration record, and test result; label unresolved items as unverified instead of presenting them as implemented contracts.

22.3. Release Checks

Privacy-policy alignment is a release gate. Before publishing the notice or enabling a material processing change, maintainers must:

- Approve and record the controller, policy version/effective date, consent versions, required registration fields, adult eligibility, and more restrictive project-specific conditions.
- Replace the supplied draft retention schedule with an approved schedule matched to actual deletion, backup, and recipient-copy capabilities; sample periods are not production commitments.
- Record provider and recipient locations, approved transfer arrangements, contractual restrictions, and any required impact assessment or sensitive-data condition.
- Confirm the policy's rights-response commitments, verify escalation and complaint contacts, and exercise access, withdrawal, erasure, recipient notification, and post-restore exclusion using synthetic records.
- Compare browser storage, telemetry, advertising/tracking statements, role access, and exports with the actual deployment. Publish changes with notice and renewed consent where required.

- Resolve the supplied policy’s downstream-model name “Jasina ASR” against the publication and handbook name “Jasinah” before publication; this policy integration does not rename the separate model project.

Before publication, maintainers must:

1. Verify the canonical repository URL, supported release tag/commit, exact tool versions, actual paths, and command names against a clean checkout; replace any remaining release-specific example markers.
2. Execute Listings 1–4 from a clean checkout and correct every undocumented step.
3. Generate or compare the schema reference, RPC signatures, database types, environment template, and RLS matrix with the deployed release.
4. Confirm that diagrams match current component boundaries, states, storage paths, and authorization flows.
5. Run the complete automated test and production-build pipeline and retain its result with the release.
6. Test deployment, forward recovery, backup restoration, and the most important troubleshooting procedures in staging.
7. Update Table 2, the changelog, known limitations, and architecture decision records.
8. Cross-check shared facts against the companion JasinaHub publication, including product/model naming, repository and privacy URLs, collection modes, configurable limits, lock and skip behavior, transcription notation, two-stage quality control, pilot purpose/eligibility/dialects/dates/totals, completed-pair counts, consent rights, funding, limitations, and the external *Jasinah* model boundary.
9. Run the documented `latexmk` command, resolve undefined references and material layout warnings, and review diagrams and screenshots at normal print scale.

Appendix A records the accepted v1.0 decisions covering Supabase adoption, separate role storage, transcriber capability, configurable pessimistic locking, audio format and duration, offline recovery, prompt retirement, anonymous feedback, export authorization, and spontaneous/read-speech modes. New decisions should use the same context–decision–consequences–alternatives structure. Superseded ADRs remain in the historical record and should link to the replacement decision.

23. System Workflows and Behaviour

JasinaHub supports three primary workflows. The table below directs readers to the detailed rules; the worked example then follows a single recording across those workflows.

Table 16: Workflow summary and authoritative documentation.

Workflow	Lifecycle summary	Authoritative detail
Speech contribution	Verify account and consent; present a spontaneous or read-speech prompt; record, preview, recover if interrupted, upload, and place the response into moderation.	Recording pipeline (Section 13), offline recovery (Section 16), domain invariants, and backend contracts (Section 14).
Transcription	Approve the transcriber and guidelines; atomically reserve one eligible recording; listen, draft, submit, and release or expire the lease without competing edits.	Concurrency (Section 12), transcription conventions and validation (Section 18.2), and backend contracts (Section 14).
Administrative oversight	Manage accounts and prompts; review recordings and transcripts; monitor activity; manage feedback; and export accepted pairs with pseudonymous identifiers.	Roles and authorization, security (Section 17), quality assurance (Section 18), and the companion <code>operations-runbook.pdf</code> .

Section 19 records the pilot settings, dates, exceptions, and outcomes.

23.1. Worked Example: One Recording from Capture to Export

This illustrative scenario uses the documented workflow, not a participant record or a test result. Suppose a verified volunteer who has consented records a 20-second answer to a health-related prompt. The prompt is available and the clip is within the deployment’s duration limit.

1. **Record and preview.** The volunteer records an answer and listens to it before submitting. The browser holds the WebM/Opus audio and saves a recovery copy before starting the upload.
2. **Recover an interrupted upload.** The connection drops before the upload completes. On the volunteer’s next visit, `RecoveryDialog` offers a retry from the retained copy. The retry must avoid creating a duplicate submission. Once the server confirms both the audio object and its database record, the client may clear the recovery copy. The saved `voice_responses` row has review status `pending`.
3. **Review the recording.** An administrator listens to the audio and checks the recording criteria. In this example, the answer is usable and contains no direct identifiers, so the administrator marks it `accepted`. It can now enter the transcription queue.
4. **Claim and transcribe.** An approved transcriber requests work. The claim RPC selects this recording and reserves it with a lease. The transcriber listens to the full clip, writes a verbatim transcript, and submits before expiry. Submission saves the text and releases the lock. The recording’s review status remains `accepted`; transcription completion is tracked separately, not as another value of that status field.
5. **Validate and export.** An administrator compares the transcript with the audio and accepts the text. Provided consent remains valid and the other export checks pass, the pair can be included in an administrator-generated ZIP and CSV manifest. The manifest links the transcript to the audio and uses a stable pseudonymous contributor identifier instead of a name.

If the lease expires before step 4 completes, the transcriber keeps the draft but must obtain a valid claim before submitting. If an audio download fails during step 5, a metadata-only CSV must be labelled incomplete. Neither recovery path bypasses the ownership or export checks described in the preceding chapters.

24. User Interface and Experience Design

The interface separates recording, transcription, and administrative tasks. Recording prompts distinguish open-response questions from read-aloud sentences so contributors know whether wording is prescribed. The dashboard shows category contributions as duration or percentage according to the configured task.

Recording and recovery. Desktop capture uses `AudioWaveform`; the mobile view uses `MinimalRecorder` for compact controls. `SegmentedProgress` displays contributions, while `RecoveryDialog` and `OfflineIndicator` expose interrupted work and connectivity. A confirmation dialog lets contributors reconsider a skip. Haptic feedback, swipe navigation, and pull-to-refresh use `useHapticFeedback`, `useSwipeGesture`, and `usePullToRefresh`, respectively.

Transcription. The workspace keeps the prompt and audio controls beside the text field. It supports replay, playback-speed adjustment, completion-gated text entry, skip, and draft recovery. Section 18.2 describes the review workflow and transcription conventions.

Presentation and accessibility. Dark and light themes use `next-themes`; translucent cards and backdrop blur provide the visual treatment. A “Skip to content” link, visible keyboard focus, and semantic control names support navigation. Keyboard operation and contrast must be tested against a stated WCAG target; individual features do not establish conformance.

25. Future Extensions

- Expanded language coverage beyond Kikuyu, with administrator-configurable dialect taxonomies.
- Sociolinguistic metadata expansion (region, education, multilingual background) gated by additional consent.

- Optional integration with semi-automated transcription tools (forced alignment, ASR pre-fill) to accelerate annotation.
- Completing and systematically evaluating the separate *Jasinah* Kikuyu ASR model on governed JasinaHub exports, including recognition benchmarks and analysis of corpus-quality effects, without coupling model code to the collection platform.
- Pluggable export formats targeting specific machine-learning frameworks (HuggingFace Datasets, Kaldi, NeMo manifests).
- Configurable retirement thresholds and prompt-authoring tools for non-technical curators.
- Contributor engagement analytics including streaks, leaderboards, and lightweight gamification.
- Inter-annotator agreement measurement for transcription consistency using established agreement methodology [5].

These extensions are compatible with the stated architectural direction, but each requires an architecture, privacy, migration, and operational-impact review before implementation.

26. Conclusion

The Kikuyu pilot demonstrated an end-to-end path from community recording to reviewed speech–transcription pairs and administrator-controlled export. Section 19 reports the outcomes, and Section 20.1 explains the limits of that evidence.

Maintainers should keep this guide aligned with the released source code, database migrations, configuration, and test results. Before accepting contributions in a new deployment, they should verify access permissions, test the rules for saving and updating records, review the community’s collection requirements, and exercise recovery. The companion operations runbook provides deployment and recovery procedures.

27. Project Attribution and Declarations

Braxton Mandara: Conceptualization, software, and writing—original draft. **Mary Kariuki:** Supervision and validation. **Ciira wa Maina:** Conceptualization, project administration, and writing—review and editing.

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work described in this document.

28. Acknowledgement

This work was conducted as part of the Artificial Intelligence for Development (AI4D) program, with the financial support of the UK government through its Global Research and Technology Development portfolio as well as Canada’s International Development Research Centre (IDRC).

A. Architecture Decision Records

The following Architecture Decision Records (ADRs) preserve the rationale behind consequential release v1.0 choices. They were documented retrospectively on 29 July 2026 and most recently reconciled with the JasinaHub publication on 4 September 2026; the original implementation decisions predate these records. A future change that reverses one of these decisions should add a superseding ADR rather than silently rewriting the historical record.

A.1. ADR-001: Adopt Supabase as the Backend-as-a-Service Platform

Status	Accepted for v1.0.
Context	JasinaHub requires authentication, relational persistence, object storage, realtime communication, and specialised server-side functions without maintaining a dedicated application-server fleet.

Decision	Use Supabase to provide GoTrue authentication, managed PostgreSQL, RLS-aware APIs and RPCs, Realtime channels, object storage, and Deno Edge Functions.
Consequences	Infrastructure and deployment overhead are reduced and critical rules remain close to PostgreSQL, but the platform acquires provider-specific configuration, service limits, and migration considerations.
Alternatives	A custom Node.js backend, independently managed PostgreSQL and object storage, or another BaaS provider.

A.2. ADR-002: Store Privileged Roles Separately from User Profiles

Status	Accepted for v1.0.
Context	Profiles contain user-editable or user-associated demographic and workflow fields; storing administrative privilege in the same update surface increases privilege-escalation risk.
Decision	Store <code>admin</code> and <code>user</code> assignments in <code>user_roles</code> , separate from <code>profiles</code> , and evaluate them through the controlled <code>has_role()</code> function.
Consequences	Role mutation can be restricted and audited independently, while ordinary authorization queries require an additional table or function lookup. The <code>SECURITY DEFINER</code> function requires a safe owner, fixed <code>search_path</code> , narrow grants, and dedicated tests.
Alternatives	A mutable role column on <code>profiles</code> , authentication-token metadata, or a frontend-only role check.

A.3. ADR-003: Model Transcriber Access as an Approved Profile Capability

Status	Accepted for v1.0.
Context	A transcriber remains a volunteer and may also contribute recordings; transcription access additionally depends on administrator approval and guideline acceptance.
Decision	Represent transcription eligibility through <code>profiles.transcription_approved</code> and guideline-acceptance fields rather than adding a third value to <code>app_role</code> .
Consequences	One account can retain volunteer behavior while gaining a controlled secondary capability. Authorization must combine role, verification, approval, and guideline state consistently, and only administrators may change approval.
Alternatives	A dedicated <code>transcriber</code> role, a separate transcriber account type, or unrestricted transcription access for verified users.

A.4. ADR-004: Use a Configurable Pessimistic Transcription Lease

Status	Accepted for v1.0.
Context	Concurrent transcribers must not receive or edit the same recording, while abandoned browser sessions must not reserve work indefinitely.
Decision	Claim a randomly selected eligible candidate through an atomic PostgreSQL RPC that creates an exclusive configurable lease in <code>transcription_locks</code> . The current deployment defaults to ten minutes and the pilot used that value. Skip, logout, submission, explicit abandonment, or expiry releases the candidate; a skipped item is temporarily excluded only for that transcriber.
Consequences	The principal double-assignment race is addressed at the database layer and abandoned work becomes reclaimable. Submission must verify ownership and expiry, deployments must test their selected lease, and slow transcribers require draft recovery or a governed renewal design.
Alternatives	Optimistic conflict detection at submission, permanent assignment, client-side locking, or a non-configurable lease.

A.5. ADR-005: Capture Browser Audio as WebM/Opus

Status	Accepted for v1.0.
---------------	--------------------

Context	Mobile contributors require compact audio with direct browser recording and upload support, while different projects may require different clip lengths.
Decision	Use the browser <code>MediaRecorder</code> API to capture WebM containers with the Opus codec and enforce a deployment-configured duration limit; the pilot used 30 seconds.
Consequences	Opus provides efficient speech compression and avoids application-side encoding on supported browsers. Browser MIME support and the active duration setting must be tested, manifests must record the actual format, and downstream consumers may require controlled transcoding.
Alternatives	WAV/PCM, MP3, AAC, server-side conversion, or a universal fixed duration.

A.6. ADR-006: Use Bounded LocalStorage for Interrupted-Recording Recovery

Status	Accepted for v1.0 with a documented migration path.
Context	Contributors may lose connectivity between recording and confirmed upload, and the release requires a simple recovery mechanism that works across page reloads.
Decision	Serialize one bounded WebM/Opus recording as Base64 through <code>recordingStorage</code> in LocalStorage and expose it through <code>RecoveryDialog</code> on the next visit.
Consequences	The mechanism is simple and widely available but synchronous, space-inefficient, quota-limited, and vulnerable to browser eviction; it is not a general offline queue. IndexedDB is preferred before multiple queued recordings or larger offline workloads are supported.
Alternatives	Memory-only state, IndexedDB, service-worker-managed queues, or immediate upload without local recovery.

A.7. ADR-007: Retire Prompts after a Configured Unique-Response Threshold

Status	Accepted for the current collection configuration.
Context	A finite contributor pool and prompt inventory must balance repeated coverage with speaker and prompt diversity. The rule was introduced after pilot collection began, so pre-rule responses were retained and some historical prompts exceed the threshold.
Decision	Remove a prompt from normal active rotation after three distinct users have submitted responses, using question-count RPCs for availability and an authoritative submission-time integrity check. Record the threshold and its effective date as governed configuration.
Consequences	Collection is steered toward breadth and no prompt dominates contributor effort. Historical pilot counts are not bounded by three responses per prompt. Later statistical objectives may require another threshold, and concurrent submissions require backend enforcement.
Alternatives	No retirement, a larger fixed threshold, time-based retirement, or adaptive sampling based on dataset coverage.

A.8. ADR-008: Minimise Identity Linkage in the Feedback Schema

Status	Accepted for v1.0.
Context	Users should be able to critique the platform without feedback being directly attached to their application profile.
Decision	Store feedback answers without a direct <code>user_id</code> column and restrict administrative access through RLS.
Consequences	Direct database linkage is reduced and candid responses are supported, but user-specific follow-up is limited. Absolute anonymity is not guaranteed because timestamps, authentication, network, and infrastructure logs may permit correlation.
Alternatives	Identified feedback, optional identity disclosure, external survey tooling, or fully unauthenticated submission.

A.9. ADR-009: Restrict Dataset Export to Administrators

Status	Accepted for v1.0.
Context	Export packages contain sensitive voice data and curated transcriptions intended for governed research use.
Decision	Permit generation and download only through the administrator interface, omit contributor names, replace them with stable TH-XXXXXXX identifiers, and distinguish complete ZIP/CSV exports from metadata-only fallbacks.
Consequences	Centralised authorization supports consent, quality, audit, and release governance, but administrators become an operational bottleneck. Export requires strong logging, protected pseudonym generation, storage controls, and release records. Stable pseudonyms permit within-dataset linkage while voice, metadata, and response content remain potentially identifying.
Alternatives	Contributor self-service download, transcriber export access, public dataset endpoints, direct bucket access, or unlinkable per-record pseudonyms.

A.10. ADR-010: Support Structured Spontaneous and Read-Speech Prompts

Status	Accepted for v1.0.
Context	Corpus projects may require natural, speaker-formulated answers, controlled read sentences, or both.
Decision	Configure each collection activity so the interface distinguishes open-response prompts from predetermined read-speech sentences while reusing the same capture, moderation, transcription, and export pipeline.
Consequences	One deployment can collect complementary speech styles without duplicating infrastructure, but prompt mode must be preserved in metadata and exports, instructions and quality criteria must match the mode, and analyses must not merge the styles without disclosure.
Alternatives	Spontaneous speech only, read speech only, or separate applications for each mode.

B. Glossary

This glossary defines the terminology used throughout the JasinaHub documentation. Implementations, user-interface labels, database comments, tests, and support procedures should use these meanings consistently.

Accepted recording

A `voice_responses` row whose audio has passed administrator quality review and is eligible for transcription and, once paired with an accepted transcription, dataset export.

Active lock

An unexpired `transcription_locks` row that grants one approved transcriber temporary exclusive responsibility for a recording.

Administrator

A user holding the `admin` value in `user_roles`; administrators perform verification, moderation, approval, validation, export, and operational tasks.

Anonymous feedback

Feedback stored without a direct application `user_id`. Infrastructure metadata may still permit indirect correlation, so the term does not imply mathematically guaranteed anonymity.

Approved transcriber

A verified volunteer whose profile has `transcription_approved = true` and who has accepted the transcription guidelines.

Canonical transcription

The single authoritative transcription associated with a voice response under the release's one-transcription-per-recording invariant.

Category

A thematic grouping of prompts, initially used to organise health-related questions.

Code-switch marker

The paired token `[cs]` used immediately before and after a non-Kikuyu word in a Kikuyu pilot transcription, for example `[cs]hospital[cs]`.

Contributor

A volunteer who submits a voice response. The terms contributor and recording volunteer refer to the same operating role in this guide.

Dataset export

An administrator-authorized package containing eligible audio and a metadata manifest; it may be a complete ZIP/CSV package or a clearly identified metadata-only fallback.

Edge Function

A server-side Deno function deployed through Supabase for specialised operations such as password reset or legacy storage migration.

Inaudible marker

The token `[?]` used when a segment cannot be heard or understood reliably and must not be guessed.

Metadata-only export

A fallback CSV produced when one or more audio objects cannot be retrieved. It is incomplete and must not be represented as a complete audio dataset.

Moderation

Administrator review of a submitted recording, resulting in an accepted or rejected status.

Prompt or question

Configured text displayed to a volunteer either to elicit a natural spoken response or to provide a predetermined sentence for read speech. These terms do not refer to feedback questions unless explicitly qualified.

Prompt retirement

Removal of a prompt from normal active rotation after the configured number of unique contributors has responded.

Pseudonymous contributor identifier

A stable project-generated export identifier in the form `TH-XXXXXXX`. It replaces a contributor name in governed exports but does not make the associated voice anonymous.

Read speech

Audio produced by speaking a predetermined sentence displayed by the platform, as distinct from an open response formulated by the contributor.

Recovery copy

A bounded Base64 copy of an interrupted recording retained by release v1.0 in `LocalStorage` through `recordingStorage`.

Rejected recording

A voice response that failed administrator quality review and is excluded from transcription and export.

RLS

PostgreSQL Row-Level Security, which filters or rejects table operations according to the authenticated identity, role, ownership, and account state.

RPC

A PostgreSQL function invoked through the Supabase remote-procedure-call interface.

Signed URL

A time-limited object-storage URL generated for an authorised reader; it should not be persisted as the canonical object identifier.

Transcription validation

Administrator review of submitted transcription text and its associated audio before the pair becomes export eligible.

Jasinah

A separate downstream Kikuyu ASR model project that consumes governed JasinaHub data exports; it is not part of the JasinaHub v1.0 codebase or deployment boundary.

Verified user

An authenticated user whose profile has been approved by an administrator for protected collection workflows.

Voice response

The database record and associated audio object created when a volunteer submits a spoken answer to a prompt.

Volunteer

The default authenticated participant role responsible for recording prompt responses.

References

- [1] David Ifeoluwa Adelani, Jade Abbott, Graham Neubig, Daniel D’souza, Julia Kreutzer, Constantine Lignos, Chester Palen-Michel, Happy Buzaaba, Shruti Rijhwani, Sebastian Ruder, et al. MasakhaNER: Named entity recognition for african languages. *Transactions of the Association for Computational Linguistics*, 9:1116–1131, 2021. doi: 10.1162/tacl_a_00416. URL <https://aclanthology.org/2021.tacl-1.66/>.
- [2] Benjamin Akera, Evelyn Nafula Ouma, Gilbert Yiga, Patrick Walukagga, Phionah Natukunda, Trevor Saaka, Solomon Nsumba, Lilian Teddy Nabukeera, Joel Tibabwetiza Muhanguzi, Imran Sekalala, Nimpamya Janat Namara, Engineer Bainomugisha, Ernest Mwebaze, and John Quinn. Sunflower: A new approach to expanding coverage of african languages in large language models. In *Proceedings of the AI for African Languages Conference 2025*, volume 314 of *Proceedings of Machine Learning Research*, pages 1–20, 2026. URL <https://proceedings.mlr.press/v314/akera26a.html>.
- [3] Benjamin Akera, Patrick Walukagga, Gilbert Yiga, John Quinn, and Ernest Mwebaze. How much speech data is necessary for asr in african languages? an evaluation of data scaling in kinyarwanda and kikuyu. In *Proceedings of the AI for African Languages Conference 2025*, volume 314 of *Proceedings of Machine Learning Research*, pages 41–49, 2026. URL <https://proceedings.mlr.press/v314/akera26b.html>.
- [4] Rosana Ardila, Megan Branson, Kelly Davis, Michael Henretty, Michael Kohler, Josh Meyer, Reuben Morais, Lindsay Saunders, Francis M. Tyers, and Gregor Weber. Common voice: A massively-multilingual speech corpus. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4218–4222, 2020. URL <https://aclanthology.org/2020.lrec-1.520/>.
- [5] Ron Artstein and Massimo Poesio. Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008. doi: 10.1162/coli.07-034-R2. URL <https://aclanthology.org/J08-4004/>.
- [6] Arun Babu, Changhan Wang, Andros Tjandra, Kushal Lakhotia, Qiantong Xu, Naman Goyal, Kritika Singh, et al. XLS-R: Self-supervised cross-lingual speech representation learning at scale. In *Interspeech 2022*, pages 2278–2282, 2022. doi: 10.21437/Interspeech.2022-143. URL https://www.isca-archive.org/interspeech_2022/babu22_interspeech.html.
- [7] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems*, volume 33, pages 12449–12460, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/92d1e1eb1cd6f9fba3227870bb6d7f07-Abstract.html>.

- [8] Etienne Barnard, Marelle Davel, and Charl van Heerden. The lwazi project: Speech-based access to information for south african languages. *South African Computer Journal*, 42:1–16, 2009.
- [9] Emily M. Bender and Batya Friedman. Data statements for natural language processing: Toward mitigating system bias and enabling better science. *Transactions of the Association for Computational Linguistics*, 6:587–604, 2018. doi: 10.1162/tacl_a_00041. URL <https://aclanthology.org/Q18-1041/>.
- [10] Andreas Bjørn-Hansen, Tim A. Majchrzak, and Tor-Morten Grønli. Progressive web apps: The possible web-native unifier for mobile development. In *Proceedings of the 13th International Conference on Web Information Systems and Technologies*, pages 344–351, 2017.
- [11] Steven Bird, Florian R. Hanke, Oliver Adams, and Haejoong Lee. Aikuma: A mobile app for collaborative language documentation. In *Proceedings of the 2014 Workshop on the Use of Computational Methods in the Study of Endangered Languages*, pages 1–5, 2014. doi: 10.3115/v1/W14-2201. URL <https://aclanthology.org/W14-2201/>.
- [12] Alexis Conneau, Min Ma, Simran Khanuja, Yu Zhang, Vera Axelrod, Siddharth Dalmia, Jason Riesa, Clara Rivera, and Ankur Bapna. FLEURS: Few-shot learning evaluation of universal representations of speech. In *2022 IEEE Spoken Language Technology Workshop*, pages 798–805, 2023. doi: 10.1109/SLT54892.2023.10023141. URL <https://doi.org/10.1109/SLT54892.2023.10023141>.
- [13] Nicole Davila, Igor Wiese, Igor Steinmacher, Lucas Lucio da Silva, Andre Kawamoto, Gilson Jose Peres Favaro, and Ingrid Nunes. An industry case study on adoption of AI-based programming assistants. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice*, pages 92–102. Association for Computing Machinery, 2024. doi: 10.1145/3639477.3643648.
- [14] David M. Eberhard, Gary F. Simons, and Charles D. Fennig. *Ethnologue: Languages of the World*. SIL International, Dallas, Texas, 28 edition, 2025. URL <https://www.ethnologue.com/>.
- [15] AbdelRahim A. Elmadany, Sang Yun Kwon, Hawau Olamide Toyin, Alcides Alcoba Inciarte, Hanan Aldarmaki, and Muhammad Abdul-Mageed. Voice of a continent: Mapping Africa’s speech technology frontier. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11028–11050. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.559. URL <https://aclanthology.org/2025.emnlp-main.559/>.
- [16] Elodie Gauthier, Laurent Besacier, Sylvie Voisin, Michael Melese, and Uriel Pascal Elingui. Collecting resources in sub-saharan african languages for automatic speech recognition: A case study of Wolof. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation*, pages 3863–3867, 2016. URL <https://aclanthology.org/L16-1611/>.
- [17] Elodie Gauthier, David Blachon, Laurent Besacier, Guy-Noël Kouarata, Martine Adda-Decker, Annie Rialland, Gilles Adda, and Grégoire Bachman. LIG-Aikuma: A mobile app to collect parallel speech for under-resourced language studies. In *Interspeech 2016*, pages 381–382, 2016. URL https://www.isca-archive.org/interspeech_2016/gauthier16_interspeech.html.
- [18] Jim Gray. The transaction concept: Virtues and limitations. In *Proceedings of the Seventh International Conference on Very Large Data Bases*, pages 144–154, 1981.
- [19] Joseph M. Hellerstein, Jose Faleiro, Joseph E. Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. Serverless computing: One step forward, two steps back. In *Proceedings of the 9th Biennial Conference on Innovative Data Systems Research*, 2019. URL <https://www.cidrdb.org/cidr2019/papers/p119-hellerstein-cidr19.pdf>.
- [20] S. H. Imam, B. Sani, D. K. Gete, B. Y. Ahamed, I. S. Ahmad, I. Abdulmumin, S. M. Yimam, M. Y. Bello, and S. H. Muhammad. Automatic speech recognition for african low-resource languages: Challenges and future directions. *arXiv preprint arXiv:2505.11690*, 2025. doi: 10.48550/arXiv.2505.11690. URL <https://arxiv.org/abs/2505.11690>.
- [21] Kenya National Bureau of Statistics. 2019 kenya population and housing census, volume iv: Distribution of population by socio-economic characteristics. Technical report, Kenya National Bureau of Statistics, Nairobi, Kenya, 2019. URL <https://www.knbs.or.ke/reports/kenya-census-2019/>.

- [22] Jonathan Mukiibi, Andrew Katumba, Joyce Nakatumba-Nabende, Ali Hussein, and Joshua Meyer. The makerere radio speech corpus: A luganda radio corpus for automatic speech recognition. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 1945–1954, 2022. URL <https://aclanthology.org/2022.lrec-1.208/>.
- [23] Wilhelmina Nekoto, Vukosi Marivate, Tshinondiwa Matsila, Timi Fasubaa, Taiwo Fagbohunge, Solomon Oluwole Akinola, Shamsuddeen Hassan Muhammad, et al. Participatory research for low-resourced machine translation: A case study in african languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2144–2160, 2020. doi: 10.18653/v1/2020.findings-emnlp.195. URL <https://aclanthology.org/2020.findings-emnlp.195/>.
- [24] Vineel Pratap, Qiantong Xu, Anuroop Sriram, Gabriel Synnaeve, and Ronan Collobert. MLS: A large-scale multilingual dataset for speech research. In *Interspeech 2020*, pages 2757–2761, 2020. doi: 10.21437/Interspeech.2020-2826. URL https://www.isca-archive.org/interspeech_2020/pratap20_interspeech.html.
- [25] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 28492–28518, 2023. URL <https://proceedings.mlr.press/v202/radford23a.html>.
- [26] Republic of Kenya. Data protection act, no. 24 of 2019. Kenya Law, 2019. URL <https://new.kenyalaw.org/akn/ke/act/2019/24/eng@2022-12-31>.
- [27] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.
- [28] Tanja Schultz. Globalphone: A multilingual speech and text database developed at karlsruhe university. In *Proceedings of the 7th International Conference on Spoken Language Processing*, pages 345–348, 2002.
- [29] The PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies*. The PostgreSQL Global Development Group, 2024. URL <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>.
- [30] Natalia Tomashenko, Xin Wang, Emmanuel Vincent, Jose Patino, Brij Mohan Lal Srivastava, Paul-Gauthier Noé, Andreas Nautsch, Nicholas Evans, et al. The VoicePrivacy 2020 challenge: Results and findings. *Computer Speech & Language*, 74:101362, 2022. doi: 10.1016/j.csl.2022.101362. URL <https://doi.org/10.1016/j.csl.2022.101362>.
- [31] Jean-Marc Valin, Koen Vos, and Timothy B. Terriberry. Definition of the opus audio codec. RFC 6716, Internet Engineering Task Force, 2012. URL <https://www.rfc-editor.org/rfc/rfc6716>.
- [32] Barack Wanjawa, Lilian Wanzare, Florence Indede, Owen McOnyango, Edward Ombui, and Lawrence Muchemi. Kencorpus: A kenyan language corpus of swahili, dholuo and luhya for natural language processing tasks. *Journal for Language Technology and Computational Linguistics*, 36(2):1–27, 2023. URL <https://aclanthology.org/2023.jlcl-2.1/>.
- [33] Lilian Wanzare, Cynthia Amol, Ezekiel Maina, Nelson Odhiambo, Hope Kerubo, Leila Misula, Vivian Oloo, Rennish Mboya, Edwin Onkoba, Edward Ombui, Joseph Muguro, Ciira wa Maina, Andrew Kipkebut, Alfred Omondi Otom, Ian Ndung’u Kang’ethe, Angela Wambui Kanyi, and Brian Gichana Omwenga. AfriVoices-KE: A multilingual speech dataset for kenyan languages. *arXiv preprint arXiv:2604.08448*, 2026. URL <https://arxiv.org/abs/2604.08448>.
- [34] World Wide Web Consortium. *MediaStream Recording*. W3C, 2026. URL <https://www.w3.org/TR/mediastream-recording/>. Working Draft, 16 March 2026.